



**Politecnico
di Torino**

Politecnico di Torino

Computer Engineering

Academic Year 2025/2026

Graduation Session July 2026

Implementation of a data processing workflow with end-to-end lineage

Supervisor:

Marco Torchiano

Candidate:

Mattia Domizio

Acknowledgements

I would like to thank Professor Torchiano, my parents Claudia and Massimiliano for allowing me to undertake this journey, my girlfriend Gaia for always supporting and putting up with me during these years of study, my tutor Edgal for guiding me through this experience, the colleagues I met at the company and all the friends who stood by me during this phase of my life.

Table of Contents

List of Figures	VIII
1 Introduction	1
1.1 Context and motivations	2
1.2 DataHub Core	3
1.3 AI agents and MCP server	4
1.4 Objective of the thesis	5
1.5 Structure of the thesis	6
2 Architecture of Teseo	8
2.1 Architectural requirements	9
2.2 Adopted design principles	10
2.3 Infrastructural stack of DataHub Core	11
2.3.1 General structure of the architecture	11
2.3.2 Client Tier	13
2.3.3 App Tier	13
2.3.4 Persistence Tier	13
2.3.5 Internal path of the metadata	14
2.4 Metadata model and graph	14
2.4.1 Entities, URN and aspects	15
2.4.2 Entity Registry and governability of the model	15
2.4.3 Relationships and reading of the lineage	16
2.4.4 Versioning and temporal coherence	17
2.4.5 Metadata event cycle (MCP and MCL)	18
2.5 Multi-source ingestion layer	18
2.5.1 Supported integration methods	19
2.5.2 Implementation pattern in Teseo	21
2.6 Exposure Layer: UI, GraphQL and API	21
2.6.1 UI for navigation	21
2.6.2 GraphQL for querying	22
2.6.3 SDK and API for automation	22

2.7	Security and observability	22
2.8	Metadata lifecycle in the Teseo architecture	23
2.9	Risk scenarios and mitigation strategies	24
2.9.1	Risk of lineage incompleteness	24
2.9.2	Risk of semantic drift	25
2.9.3	Risk of regression	25
2.10	Observed architectural benefits	25
2.11	Positioning of the evolutionary modules	26
3	Multi-source ingestion and end-to-end lineage	27
3.1	Snowflake: DDL and setup	27
3.1.1	Initial setup and permission validation	28
3.1.2	DDL deployment of the domain and of the procedures	29
3.2	DataHub configuration	36
3.3	Snowflake ingestion	36
3.4	dbt ingestion and connection with Snowflake	39
3.4.1	dbt project setup and artifact generation	39
3.4.2	dbt recipe and noise control in the graph	40
3.4.3	Snowflake-dbt bridge reconciliation	41
3.5	Power BI ingestion: from local mock to service	43
3.5.1	Motivation for the local model	43
3.5.2	Generation and ingestion scripts	44
3.5.3	Connection to Power BI Service	45
3.6	Construction of the end-to-end lineage	46
3.7	Example of ingestion from other sources	48
3.7.1	Databricks case (Unity Catalog)	48
3.7.2	The true value of Teseo	49
4	Data Governance	50
4.1	Data Governance and metadata	50
4.1.1	What Data Governance is	50
4.1.2	Main dimensions of governance	51
4.2	Role of governance in Teseo	52
4.3	Governance entities in Teseo	52
4.3.1	Tagging and classification	52
4.3.2	Domains and ownership	53
4.3.3	Business Glossary and Structured Properties	54
4.3.4	Assertions and Data Contracts	54
4.3.5	Incident management	55

5	MCP Server and AI tools	57
5.1	Model Context Protocol	57
5.1.1	Data layer and transport layer	58
5.1.2	Session lifecycle	59
5.1.3	Security and trust boundaries	60
5.1.4	MCP in data-centric contexts	61
5.2	DataHub MCP Server	61
5.2.1	Deployment	61
5.2.2	Catalog of tools	62
5.3	Custom write-oriented tools	62
5.3.1	Structure of the file	63
5.3.2	Non-destructive merge	63
5.3.3	Types of tools	64
5.3.4	Operational detail of the tool families	65
5.4	Power BI tools	66
5.4.1	Operational flow	66
5.4.2	Generation of a PBIP project	67
5.4.3	Validation, estimation and packaging	67
5.4.4	Reverse engineering and conversion between formats	68
5.4.5	Implications for Teseo	69
5.5	Effectiveness metrics and production adoption	70
5.5.1	Technical and governance KPIs	70
5.5.2	Adoption model of the Teseo project	71
5.5.3	Observability and interpretation of the KPIs	72
5.6	Limitations	72
6	Results and evaluations	74
6.1	Size of the problem and of the solution	74
6.1.1	Size of the problem	74
6.1.2	Size of the solution	75
6.2	Evaluation of the multi-source ingestion	77
6.2.1	Coverage of the catalog	77
6.2.2	Ingestion times and idempotence	78
6.2.3	End-to-end lineage	78
6.3	Evaluation of the metadata governance	80
6.3.1	Semantic enrichment and coverage	80
6.3.2	Quality and monitoring of the assets	81
6.4	Evaluation of the MCP tools	81
6.4.1	Technical KPIs of the agent	81
6.4.2	Governance KPIs	82
6.5	Evaluation of the Power BI pipeline	83

6.6	Overall analysis of the objectives achieved	83
6.7	Limitations emerged in the evaluation	85
7	Conclusions	86
7.1	Contributions of the thesis	86
7.2	Evolutionary directions	87
7.3	Final considerations	88
	Bibliography	91

List of Figures

1.1	Data and Metadata	2
1.2	DataHub Core layers	4
2.1	Stream-based management of metadata via Kafka	10
2.2	Three-tier architecture of DataHub Core in the Teseo deployment	12
2.3	Simplified Metadata Model	16
2.4	Metadata ingestion flow	19
2.5	Synchronous and asynchronous publication of an MCP	20
2.6	Teseo section for token management	23
3.1	Database and schema structure in Snowflake	35
3.2	Lineage result after Snowflake + dbt ingestion	42
3.3	End-to-end lineage in Teseo	47
4.1	Data Governance in Teseo	51
4.2	Life cycle of an incident generated by a failed assertion	56
5.1	MCP architecture in Teseo	58
5.2	Data layer and transport layer	59
5.3	Power BI publication flow from Teseo	69
6.1	Stateful ingestion and soft delete of obsolete assets	78

Chapter 1

Introduction

The growing complexity of enterprise data warehouses makes it increasingly difficult to reliably understand *where data comes from, how it is transformed and where it is used*. In practice, many organizations simultaneously use cloud platforms, ELT tools, BI systems and pipelines orchestrated by different tools. In the absence of a unified technological catalog, this heterogeneity produces information fragmentation, semantic duplication and poor end-to-end traceability [1].

The problem does not concern documentation alone, but the operational capacity to manage risk: without reliable lineage, a change to a source table can propagate downstream without immediate visibility, impacting KPIs, dashboards and decision-making processes. In parallel, the demands for auditability, compliance and data governance are increasing, requiring a transparent reconstruction of the dependencies between different technological layers [2]. It is precisely in this scenario that **Teseo** fits, an initiative oriented towards the development of a **multi-source metadata ingestion** and **end-to-end lineage** platform, based on **DataHub Core** (open-source, version 1.4.0) and progressively extended with AI components built on MCP servers.

The thesis stems from an ongoing project activity, carried out at the company, and documents its state of progress, the technical decisions and the results obtained. In the first phase of the work, the focus was placed on two main directions: *(i)* consolidation of the documentary and methodological foundation on DataHub Core; *(ii)* design and implementation of the **Snowflake + dbt** metadata ingestion flow, a priority requirement for extending the integrations already in place (BigQuery, Qlik and Databricks) [3, 4, 5]. After obtaining the expected results on these fronts, the thesis focused on the integration with **Power BI** and on the refinement of end-to-end lineage through AI models built on **MCP servers** [6, 7].

1.1 Context and motivations

Organizations that adopt heterogeneous data stacks frequently face fragmentation problems: incomplete catalogs, non-explicit ownership, partial lineage and difficulty in assessing the downstream impacts caused by changes to upstream datasets.

In this context, **metadata ingestion** is not an accessory activity but an infrastructural element: it makes it possible to extract, normalize and publish technical and semantic metadata coming from different systems, enabling a coherent view of the data catalog. This process reduces the time needed to understand dependencies, facilitates collaboration between teams and supports faster decisions in the presence of incidents or evolutionary changes.

The work was carried out at **Target Reply**, a company of the Reply group specialized in the development of advanced solutions in the areas of *Analytics*, *Big Data*, *Data Science*, *Cloud Architectures* and *Artificial Intelligence* [8]. In this professional context, Teseo is positioned within an end-to-end path that goes from the identification of information needs to the design and implementation of solutions for the integration, modeling and valorization of data.

The term **metadata** refers to the descriptive information that contextualizes data, offering a deeper understanding of the content and its characteristics. As illustrated in Figure 1.1, around a datum (table or column) four dimensions of metadata can be distinguished: *structure* (schema, types, keys...), business *semantics* (definitions, domains, glossary...), organizational *responsibilities* (ownership, compliance...) and *operational traces* (origin, frequency, quality...) [1]. In other words, metadata transform a set of meaningless tables into an interpretable, governable and reusable information asset.

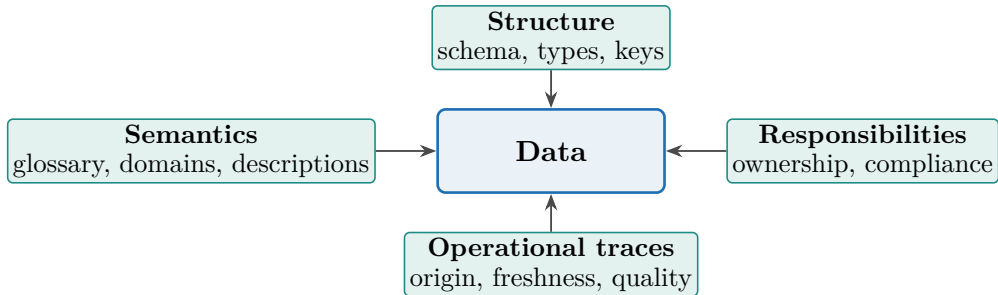


Figure 1.1: Data and Metadata

Within this framework, **lineage** represents the *dynamic dimension* of metadata,

as it describes the derivation links between entities and makes it possible to reconstruct the path that data travels along the various layers, down to the actual origin of the data itself [2]. This enables *impact analysis*, *root-cause analysis* and the *validation* of the transformations applied over time.

In this scenario, the contribution of **Teseo** is to make these elements observable in a unified way even in the presence of heterogeneous sources, not only showing the complete catalog of assets, but also the end-to-end lineage. This ensures semantic continuity and traceability of the data flow, essential requirements for analytical **reliability** and operational **governance**.

1.2 DataHub Core

DataHub Core was adopted as the backbone of the Teseo project for its ability to integrate metadata coming from different platforms into a single knowledge graph and for its *open-source* nature [9]. The choice was also guided by the availability of mature connectors, extensible APIs and a model oriented towards the evolutionary management of metadata platforms. In terms of functionality, the platform enables:

- unified cataloging of datasets, views, pipelines, dashboards and their attributes;
- search and discovery on heterogeneous assets;
- representation of lineage at the table level and, where available, at the column level;
- support for governance (ownership, descriptions, domains, tagging and quality).

The approach we decided to adopt in the Teseo project was **progressive**:

- *consolidation of ingestion and lineage on priority sources*: in this phase, we focused our attention on Snowflake (a data warehouse platform widely adopted by clients) and on dbt (an orchestration tool that makes it possible to build a detailed lineage at the model level and to manage complex transformation processes); the choice of dbt was also guided by its ability to integrate seamlessly with Snowflake [3, 4]. The objective was therefore to ensure a solid base of technical and semantic metadata, with a reliable and easily navigable lineage;
- *extension towards analytical consumption levels*: after consolidating the technical base, we extended coverage towards business intelligence tools such as Power BI, with the aim of enabling an end-to-end lineage that reaches the level closest to the customer [6]. This step was crucial to make metadata truly

useful to end users, facilitating the understanding of dependencies and impact analysis through AI-powered tools (and not only).

Figure 1.2 summarizes the role of DataHub as a unifying layer: the connectors collect metadata from the various sources, the Ingestion Framework normalizes and publishes them to the Metadata Service, which makes them persistent and queryable through a single catalog.

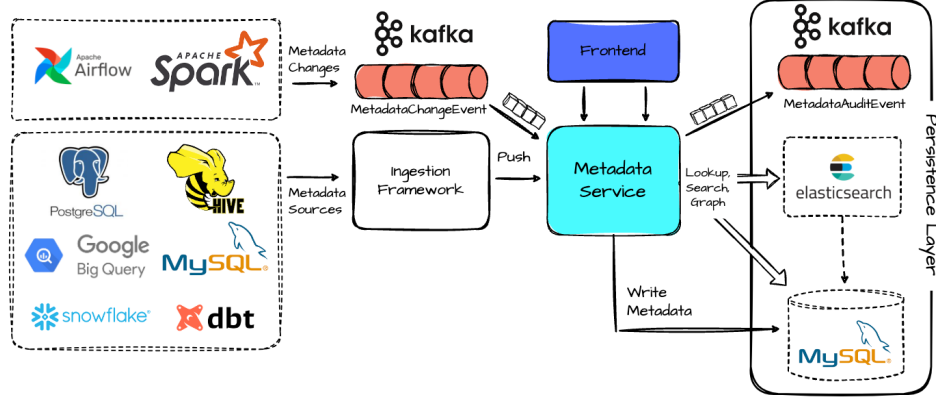


Figure 1.2: DataHub Core layers

Source: DataHub official documentation [10]

1.3 AI agents and MCP server

Beyond the pure cataloging of metadata, the project envisages the adoption of AI agents capable of querying and acting on the metadata catalog in a controlled way. In this scenario, the **Model Context Protocol (MCP)** plays a central role as it provides a standard interface through which models can access external tools and contexts, maintaining a separation between the agent’s logic and the metadata sources [7, 11].

In the case of DataHub Core, the use of an MCP server makes it possible to build agents oriented to concrete data governance use cases, such as:

- contextual search for datasets and owners (*ownership discovery*);
- navigation of upstream/downstream lineage;
- impact analysis downstream following a change to columns and dependencies made upstream;

- technical assistance in understanding the various transformations applied to fields.

This approach makes it possible to integrate the potential of AI models with the informational richness of the metadata catalog, while maintaining rigorous control over the operations performed and the traceability of interactions. In particular, the adoption of the MCP protocol makes it possible to introduce application *guardrails* and a clear separation of responsibilities between the conversational level, the orchestration level and the data level.

1.4 Objective of the thesis

The general objective of the thesis is to describe the design, implementation and progressive validation of the Teseo platform, with particular attention to:

- adoption of DataHub Core 1.4.0 as the core of the metadata platform;
- implementation of the Snowflake connector and its ingestion pipeline;
- integration of dbt to enable model-to-model lineage and semantic context;
- definition of an architectural base extensible towards AI functionality;
- preparation of the integration with BI tools (in particular Power BI) for an end-to-end lineage that is more readable and useful to the business.

Beyond the technical aspects, the thesis intends to show concretely which business criticalities are solved by the adoption of Teseo and which operational benefits derive from it. In particular, the platform is designed to intervene on four recurring problems in data-driven organizations:

- *poor visibility of the data assets*: metadata are often distributed across different tools and difficult to consult in a uniform way;
- *incomplete or fragmented lineage*: without an end-to-end view of dependencies, the impact analysis of a change remains partial and risky;
- *non-systematic governance*: ownership, domains, classifications and policies are frequently not standardized;
- *low productivity in analysis activities*: data engineers, analysts and stakeholders spend excessive time reconstructing the context, origin and meaning of data.

From this perspective, Teseo is not treated simply as a huge catalog, but as an **organizational facilitator** capable of improving decision quality, collaboration and speed of execution. The main expected benefits are:

- reduction of *impact analysis* and *root-cause analysis* times;
- increased reliability of analytical flows thanks to more complete and coherent metadata;
- greater transparency towards business and governance functions through shared taxonomies (tags, domains, ownership);
- acceleration of technical onboarding on new sources and new projects;
- evolution towards an operating model assisted by AI agents, with controlled interactions through MCP.

A further objective therefore concerns the adoption of the MCP server to enable AI agents capable of querying and manipulating metadata in a controlled way, maintaining traceability, security and governance of operations.

1.5 Structure of the thesis

The following chapters will present the main development steps that accompanied this thesis journey, following the structure illustrated below:

- **Chapter 2 - Architecture of Teseo.** This chapter illustrates in detail the architectural structure of the platform, describing components, responsibilities, communication flows and the role of DataHub Core as the core of Teseo.
- **Chapter 3 - Multi-source ingestion.** This chapter analyzes the implementation and functioning of the ingestion flows, with particular attention to Snowflake, dbt and Power BI, highlighting how these integrations build a complete end-to-end lineage; the chapter also includes other examples of ingestion from sources such as Databricks or PostgreSQL.
- **Chapter 4 - Data Governance with Teseo/DataHub Core.** This chapter explores the use of the governance functionalities (tags, domains, ownership, documentation, asset classification...), showing how Teseo supports the control and standardization of the information assets.
- **Chapter 5 - MCP Server and AI tools.** This chapter describes the implementation and use of the MCP server to enable operations assisted by AI agents on DataHub Core/Teseo, with a focus on practical use cases and benefits in terms of productivity.

- **Chapter 6 - Results and evaluations.** This section presents an overall evaluation of the results obtained, measuring the operational gain of adopting Teseo and its positioning within business processes, with particular attention to data-driven organizations.
- **Chapter 7 - Conclusion.** This concluding chapter summarizes the contributions of the thesis, the current limitations of the solution and the main evolutionary directions of the project.

Chapter 2

Architecture of Teseo

Teseo was conceived and implemented as a **metadata intelligence platform** based on **DataHub Core**, with the aim of providing a unified, reliable and navigable view of the enterprise data catalog. This chapter describes the architecture of Teseo, highlighting the components, responsibilities and operational flows that make it possible to move from a heterogeneous set of data sources to a coherent catalog that can be queried in a systematic way [10, 12].

As already briefly announced earlier, the adopted architecture was defined this way mainly to respond to four concrete needs that emerged during the project work:

- **multi-source integration of metadata** in order to overcome information fragmentation and build a centralized catalog;
- **end-to-end traceability of dependencies** to enable more reliable impact analyses and reduce the dependence on tacit knowledge;
- **operational support for data engineers and data analysts** through a user-friendly interface and tools for navigating the metadata graph (*search function*);
- **extensibility towards new functionalities**, without however compromising the stability of the platform core.

This therefore led to a clear separation between the metadata collection level, the metadata graph management level, the metadata and lineage consultation level, and the level of integration with external tools.

2.1 Architectural requirements

The solution was developed in an **enterprise context** where **data warehouse** systems, **ETL** tools and **Business Intelligence** platforms coexist, but each of these has different operational logics. Therefore, in the absence of a central platform that acts as a *facilitator* and *orchestrator* of metadata, understanding the past dependencies among all these layers is fragmented: indeed, the same dataset can be described:

- differently depending on the **tool** used and the **operational level** (*layer*) we are in;
- with **ownership** that is not always explicit and that can vary depending on the **context** (for example: technical ownership over a transformation job, functional ownership over a dataset, consumption ownership over a dashboard...);
- with **lineage** that is often incomplete (limited to a few layers or to a few assets within a layer) and not always updated in a reliable and stable way.

To face this complex scenario in the best possible conditions, the architecture of Teseo was guided by the following **architectural requirements**:

- **Semantic unification**: representing heterogeneous assets (datasets, dashboards, jobs, policies, ownership) according to a unified model;
- **Technical traceability**: maintaining upstream/downstream relationships everywhere along with context information useful for impact analysis, data discovery and lineage navigation;
- **Operability**: allowing scheduled or on-demand ingestion with explicit and verifiable configuration, by means of the **YAML recipes** (*metadata ingestion recipe*);
- **Scalability**: supporting a growing number of sources without rewriting the application core, but simply extending it with new recipes and new **connectors**;
- **Extensibility**: integrating external APIs and tools, while preserving the coherence of the metadata model.

Following an in-depth evaluation, DataHub Core proved to be the component best suited to satisfy these requirements thanks to several characteristics:

- **schema-first model:** first a well-typed model of the metadata is defined (*JSON Schema* with types, fields, relationships...) and then the ingestion is configured to populate this model. Thanks to this *clear contract* between model and ingestion, it is possible to maintain coherence among different sources, reduce the risk of semantic ambiguity and facilitate the progressive and controlled extension of the catalog [9];
- **stream-based management** of changes to metadata: with the adoption of a Kafka broker for the propagation of metadata events, DataHub makes it possible to keep the transactional state and the queryable state synchronized, reducing the risk of misalignment between the displayed catalog and the metadata actually ingested. Figure 2.1 illustrates this flow: a change proposal (*Metadata Change Proposal*) is propagated through a Kafka topic to the Metadata Service, which updates the transactional state (MySQL) and, through the change log (*Metadata Change Log*), the queryable and indexed state (ElasticSearch);

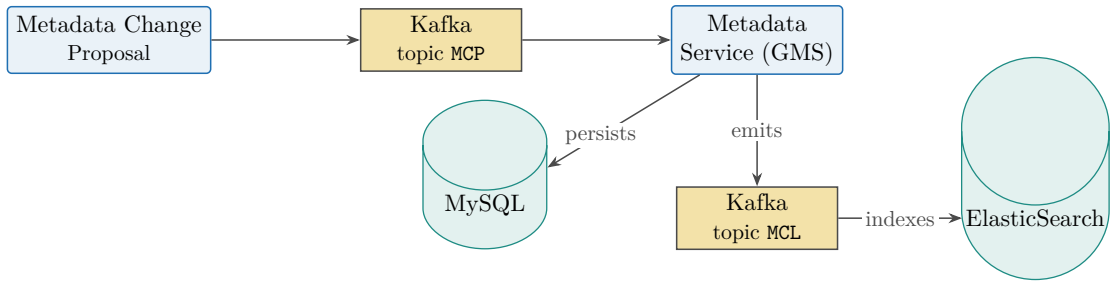


Figure 2.1: Stream-based management of metadata via Kafka

- **availability of APIs:** through DataHub Core, one can rely on APIs oriented both to human interaction (*UI* and *GraphQL*) and to automation (*Python SDK*, *Java SDK* and low-level APIs such as *OpenAPI*) [13].

2.2 Adopted design principles

The architectural definition of Teseo was not limited to the choice of components and the satisfaction of the requirements, but also followed a set of principles that guided the implementation:

- **Separation of responsibilities:** the collection of metadata, their persistence and their exposure were kept on distinct planes;
- **Explicit configurability:** each integration was described through versionable YAML recipes, reducing implicit dependencies;

- **Incremental evolution:** first the consolidation of the core stack and of the priority sources (in this specific case Snowflake), then the progressive extension to other sources and platforms;
- **Adherence to the open-source product:** the customizations were contained in order to avoid diverging too much from DataHub Core, so as to facilitate future updates and maintain compatibility with new versions.

These principles made it possible to build a robust and flexible platform, suitable for supporting data discovery, advanced governance and AI tools.

2.3 Infrastructural stack of DataHub Core

As explained before, Teseo was implemented in an on-premises environment using the Docker quickstart of DataHub 1.4.0, but suitably configured to the project's requirements: in particular, the React frontend was customized to make it suitable for the use of AI-powered tools, but authentication was also enabled by setting the `METADATA_SERVICE_AUTH_ENABLED` option to `true` in the `docker-compose.yml`. This choice made it possible to obtain a reproducible environment, quick to start and consistent with the official documentation [12, 10].

2.3.1 General structure of the architecture

The architecture of DataHub Core can be divided into 3 main tiers:

- **Client Tier;**
- **App Tier;**
- **Persistence Tier.**

Figure 2.2 shows the three tiers of the architecture adopted in Teseo, reporting in the Client Tier only the sources actually integrated in the project (Snowflake, dbt and Power BI). In the deployment used, the services are configured with authentication enabled on the metadata service side, so as to ensure controlled access to read and modify operations, including the generation of personal tokens for external integrations. All this happens thanks to the `METADATA_SERVICE_AUTH_ENABLED` variable set to `true` in the `docker-compose.yml`. Let us now look at these three tiers in more detail.

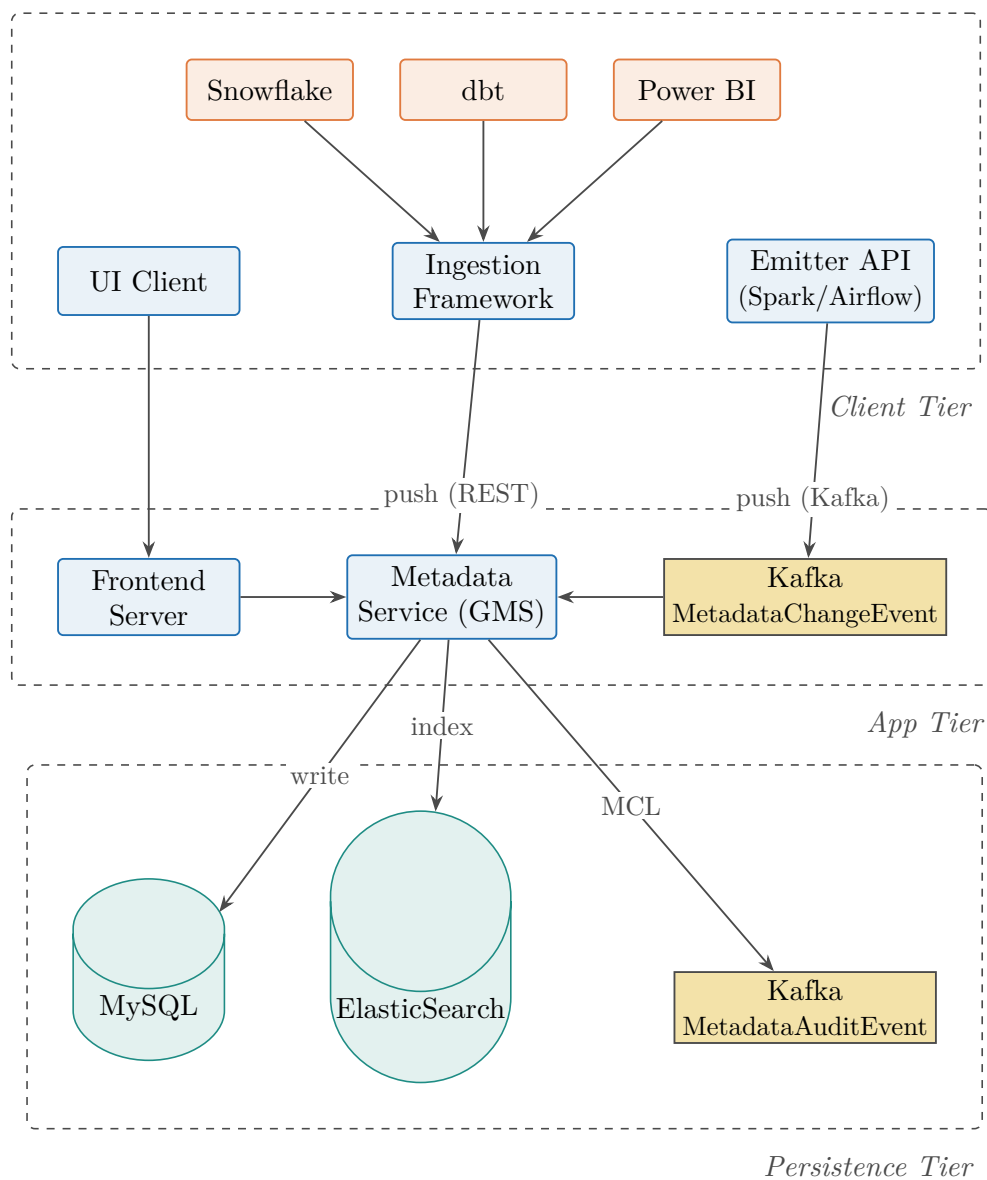


Figure 2.2: Three-tier architecture of DataHub Core in the Teseo deployment

2.3.2 Client Tier

The **Client Tier** includes the channels of interaction with users and with the ingestions from external sources. In particular, it comprises:

- **UI Client:** the main user interface for data discovery, graph navigation and operational management. It communicates with the Frontend Server of the App Tier through REST and GraphQL APIs;
- **Ingestion Framework:** the tool for collecting and normalizing the metadata received from the sources (such as BigQuery, Snowflake, PostgreSQL, Databricks, dbt, Looker, MySQL...), configurable through YAML recipes or directly from the UI. It interacts with the Metadata Service of the App Tier for metadata persistence and with the Kafka broker of the App Tier for the propagation of change events;
- **Emitter API:** it undergoes the real-time push from services such as Apache Spark and Apache Airflow, and is responsible for forwarding the events to the Kafka broker of the App Tier (*MetadataChangeEvent*).

2.3.3 App Tier

The **App Tier** includes the application services that manage the business logic and the interaction with the Persistence Tier components. In particular, it comprises:

- **Frontend Server:** it handles the requests coming from the UI Client, interacting with the Metadata Service to read and modify the metadata;
- **Metadata Service:** it manages the metadata persistence logic, receiving requests from the Frontend Server and from the ingestion tools, and interacting with the Persistence Tier components to store and index the data;
- **Kafka broker** (*MetadataChangeEvent*): it manages the propagation of metadata change events, making it possible to keep the transactional state and the queryable state synchronized, reducing the risk of misalignment between the displayed catalog and the metadata actually ingested.

2.3.4 Persistence Tier

The **Persistence Tier** includes the components responsible for the persistence and indexing of the metadata. In particular, it comprises:

- **MySQL**: a relational database that acts as the primary store for the metadata, managing the persistence of the aspects and relationships received from the Metadata Service;
- **Elasticsearch**: it manages the metadata indexing logic, receiving requests from the Metadata Service and creating indexes optimized for search and metadata discovery;
- **Kafka broker** (*MetadataAuditEvent*): it manages the propagation of audit events on the metadata, making it possible to maintain traceability and compliance.

2.3.5 Internal path of the metadata

As can be seen from the architecture described above, the external sources do not write directly to the catalog, but rather send metadata through the Ingestion Framework, which maps them onto the metadata model (*schema-first model*) and forwards them to the platform's input endpoints. From here, the persistence and indexing chain updates the state of the metadata graph and the queryable representation directly available from the UI.

Indeed, the internal path can be summarized in four steps:

1. extraction and normalization of the metadata from the source;
2. emission towards DataHub with schema validation;
3. persistence and update of the relationships;
4. indexing for search, discovery and lineage navigation.

This pipeline is what makes it possible to maintain coherence between the transactional state and the queryable state, reducing (as said previously) the risk of misalignment between the displayed catalog and the ingested metadata.

2.4 Metadata model and graph

One of the determining factors in the design of Teseo was DataHub's **Metadata Model**, based on entities, aspects and relationships. This model makes it possible to describe different assets in a homogeneous way and to link them along technical and semantic dimensions [14]. This section explores the mechanisms that make the model truly extensible over time: *Entity Registry*, *key aspects*, types of aspects and the cycle of *metadata events*.

2.4.1 Entities, URN and aspects

Each asset is identified by a stable **URN** (*Uniform Resource Name*) and belongs to a specific class of entity (for example dataset, data job, dashboard). The properties of the entity are not treated as a single block, but subdivided into aspects, each dedicated to a specific informational perspective (schema, ownership, descriptions, tags, lineage...).

This structure offers two relevant architectural advantages:

- **non-destructive evolution:** new aspects can be introduced without redefining the entire entity;
- **selective update:** even just one informational portion can be modified while keeping the other attributes unchanged.

All this made it possible to progressively increase the integrations in Teseo, since each new source can populate only the aspects within its competence without forcing global migrations of the model.

A relevant element that emerged from the analysis of the documentation is the role of the **key aspects**: these are special aspects that uniquely identify an entity and from which the related URN is derived. This mechanism makes the *primary-key lookup* in the metadata graph coherent and ensures the stability of the identifiers across successive ingestions. In particular, the format of the URN is built deterministically starting from the fields of the key aspect, thus avoiding semantic collisions between similar entities but belonging to different platforms or environments [14].

2.4.2 Entity Registry and governability of the model

Another fundamental aspect for architectural sustainability is the way in which DataHub defines its global schema: the metadata model is orchestrated through the **Entity Registry** (`entity-registry.yml`), which is responsible for the declaration of the entities and their associated aspects. The Entity Registry is validated at the startup of the *Metadata Service*.

This **declarative** type of system has two relevant practical implications:

- the extension of the model does not require destructive interventions on the core, but controlled additions of declared entities/aspects;

- the coherence between configuration and actual schema is verified in a centralized way, thus reducing modeling errors during evolution.

From a design point of view, the Entity Registry therefore represents the *Model Manager*, whose task is to balance extensibility and semantic control [14]. Figure 2.3 summarizes the model in a simplified form: an entity (for example a Dataset) is uniquely identified by a URN derived from its own key aspect and is described by independent aspects (schema, ownership, tags, glossary terms, lineage), which can be updated separately.

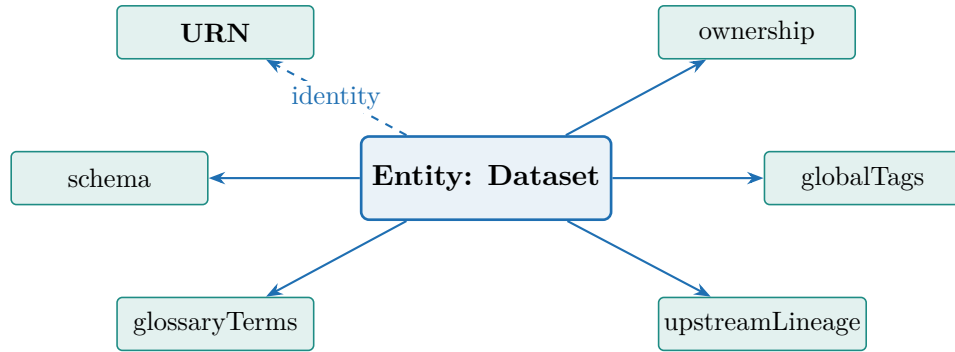


Figure 2.3: Simplified Metadata Model

2.4.3 Relationships and reading of the lineage

Having an orderly and complete catalog of the data is very important, but, without being able to visualize the relationships among these data and their dependencies, there is no real added value: **it is like having a safe full of gold, but without the combination to open it.**

The true potential of Teseo emerges in the capacity to make the relationships among the entities explicit: in particular, the upstream/downstream representation enables the reading of the dependencies between sources (for example Snowflake), transformations (for example dbt) and analytical consumption layers (for example Power BI). Indeed, as already mentioned previously, this supports more reliable impact analyses and reduces the dependence on tacit knowledge distributed across the team (thus favoring a better understanding for non-experts and new team members).

The adoption of a graph model also makes it possible to integrate different dimensions into the same lineage path: not only technical links between tables

and jobs, but also ownership, domains and documentation, creating a unified view useful both to technical profiles and to data governance stakeholders.

2.4.4 Versioning and temporal coherence

The versioning of the metadata aspects is relevant for analyzing their evolution over time (for example schema updates or changes in their properties): this system enables historical comparisons and verification of the evolution of the catalog without losing the current snapshot.

It is however necessary to distinguish **versioned aspects** and **timeseries aspects** at the operational level:

- **versioned aspects**: they represent the most recent state of an aspect, with the possibility of accessing previous versions for audit or rollback purposes;
- **timeseries aspects**: they represent events ordered in time (for example profiling and usage statistics) and can be queried by time interval.

This separation makes it possible to optimize persistence and queries according to the type of information sought [14]. For Teseo, this capacity is a key element in maintaining technical auditability during the phases of extending the ingestion pipelines, especially in cases where the verification requires both the current state of the asset and its previous versions.

Below is an example of ingestion of a timeseries aspect:

```
1 from datahub.metadata.schema_classes import (
2     ChangeTypeClass,
3     DatasetFieldUsageCountsClass,
4     DatasetUsageStatisticsClass,
5 )
6 from datahub.emitter.kafka_emitter import DatahubKafkaEmitter
7 from datahub.emitter.mcp import MetadataChangeProposalWrapper
8 from datahub.emitter.rest_emitter import DatahubRestEmitter
9
10 usageStats = DatasetUsageStatisticsClass(
11     timestampMillis=1629840771000,
12     uniqueUserCount=10,
13     totalSqlQueries=20,
14     fieldCounts=[
15         DatasetFieldUsageCountsClass(
16             fieldPath="col1",
17             count=10
18         )
19     ]
20 )
```

```

21 |
22 | mcpw = MetadataChangeProposalWrapper(
23 |     entityType="dataset",
24 |     aspectName="datasetUsageStatistics",
25 |     changeType=ChangeTypeClass.UPSERT,
26 |     entityUrn="urn:li:dataset:(urn:li:dataPlatform:hive,
27 |         SampleHiveDataset,PROD)",
28 |     aspect=usageStats,
29 | )
30 | # Instantiate appropriate emitter (kafka_emitter/rest_emitter)
31 | # my_emitter = DatahubKafkaEmitter(""<config>"")
32 | my_emitter = DatahubRestEmitter("http://localhost:8080")
33 | my_emitter.emit(mcpw)

```

Listing 2.1: Example of ingestion of a timeseries aspect

2.4.5 Metadata event cycle (MCP and MCL)

Another element that was relevant in the choice of DataHub Core is the model of the **Metadata Events**: changes to the metadata are proposed as **Metadata Change Proposals (MCP)** and, once applied to the graph, they produce **Metadata Change Logs (MCL)**. In practice, the MCP represents the request to mutate a specific aspect, while the MCL represents the change actually committed and observable downstream [15].

From an architectural point of view, this distinction clarifies the path of **reliability** towards the metadata catalog:

- the Metadata Change Proposal enables the controlled entry of the change proposal (CREATE, UPSERT, DELETE) on an aspect;
- the Metadata Change Log traces the applied change, also including the previous state and system metadata functional to the audit.

This system is very useful because it formally links the ingestion configuration to the behavior observed in the catalog, providing a clearer perimeter for troubleshooting and post-run validation [15].

2.5 Multi-source ingestion layer

The ingestion layer constitutes the point of contact between the source and the metadata platform. In DataHub, this level is supported by a modular Ingestion

Framework that makes it possible to configure the ingestions through YAML recipes, with support for different integration modes [5, 12].

As also highlighted by the official architectural documentation, the pivot of the ingestion is the **Metadata Change Proposal (MCP)** cited above: each change is modeled as a proposal to modify an aspect of the graph. This makes it possible to unify heterogeneous input channels towards a common semantic format, maintaining coherence between sources and internal model [15, 16]. Figure 2.4 summarizes this path: the sources are read by the Ingestion Framework configured through a YAML recipe, which generates the Metadata Change Proposals delivered to the Metadata Service in asynchronous mode (Kafka) or synchronous mode (HTTP) and finally persisted and indexed.

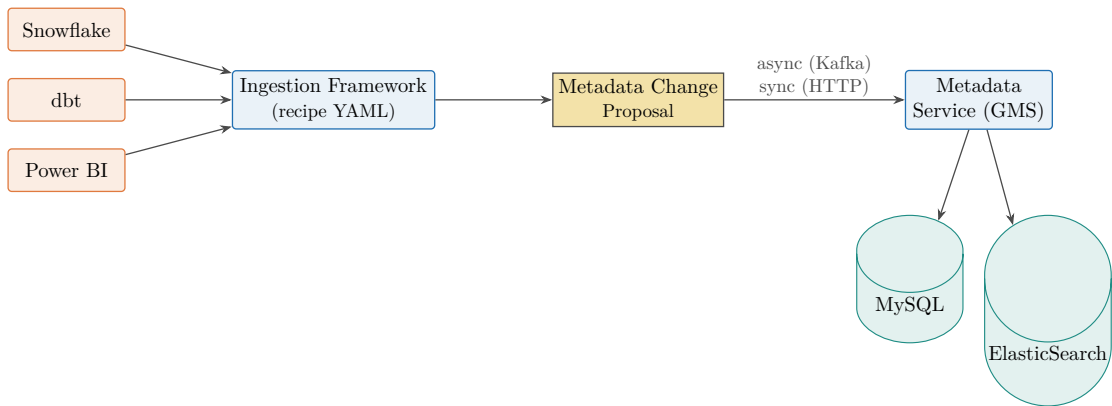


Figure 2.4: Metadata ingestion flow

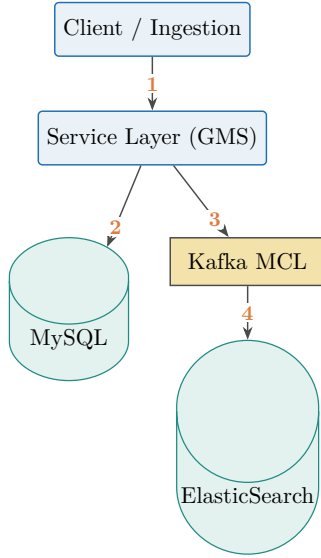
2.5.1 Supported integration methods

DataHub provides three main modes for *ingesting* metadata: configuration from the interface (*UI*), execution from the *CLI* (by means of YAML recipes; in my case the choice fell on this one because it is declarative and reproducible) and from the API (*SDK*). The coexistence of these options is useful because it makes it possible to choose the channel most suited to the use case (one option does not preclude the others).

A relevant point, not made explicit earlier, is that a Metadata Change Proposal can be published either in **asynchronous** mode (Kafka topic) or in **synchronous** mode (HTTP endpoint of the service tier). This dual mode makes it possible to balance scalability and transactional control according to the type of integration: high-volume pipelines can favor Kafka, while processes that require immediate feedback can use the HTTP channel [16]. Figure 2.5 compares the two modes:

in synchronous publication the client writes directly to the Service Layer (few steps, immediate feedback), while in the asynchronous one the proposal first passes through a Kafka topic and a dedicated consumer, introducing more steps in exchange for greater decoupling and scalability.

Synchronous publication (HTTP)



Asynchronous publication (Kafka)

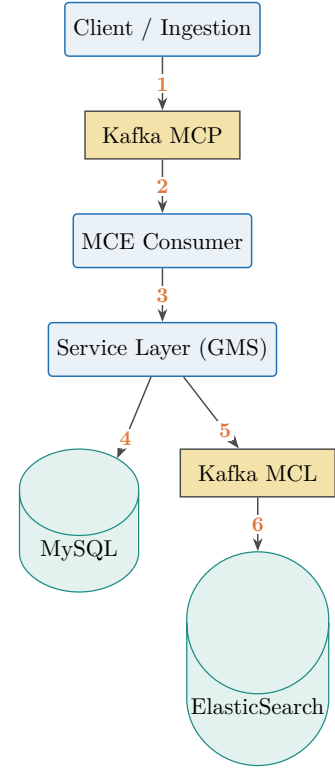


Figure 2.5: Synchronous and asynchronous publication of an MCP

The integrations can follow a *pull-based* approach (periodic collection from the source) or a *push-based* approach (emission of metadata events from the source system). Within the realized perimeter, the main strategy was the pull-based one for the priority sources (in my case mainly Snowflake) since it is more controllable in the initial phase and simpler to standardize in the operational cycles.

As seen from the architecture, in pull-based scenarios, the pipelines can also be orchestrated by external schedulers (for example Airflow or Spark), while in push-based scenarios the publication of the MCPs can occur directly through dedicated emitters [16].

2.5.2 Implementation pattern in Teseo

The pattern adopted for each new source was structured into repeatable steps:

1. definition of the informational perimeter to be extracted (assets, schema, usage, lineage, ownership...);
2. writing of the YAML recipe with connection parameters, functional options and source/sink configuration;
3. controlled execution of the ingestion in a local environment from the CLI;
4. verification in DataHub of the consistency between ingested entities, relationships and search;
5. consolidation of the operational information for subsequent executions.

This approach made it possible to reduce the difficulty in managing the new integrations and to keep the behavior of Teseo predictable during the expansion of the catalog.

As cited in the previous section, the Metadata Change Proposals are then processed by dedicated application components (*consumer jobs*) that apply them to the Metadata Service. This step is fundamental because it separates the moment of emission from the phase of persistence of the change, making the entire ingestion cycle more robust in the presence of heterogeneous sources [16].

2.6 Exposure Layer: UI, GraphQL and API

Access to the metadata in Teseo was designed according to the principle "*multiple interfaces, single source of truth*": different channels for users and automations, but a single metadata graph as a consistent informational base.

2.6.1 UI for navigation

The DataHub React UI was used as the main channel for daily discovery activities, schema consultation, ownership verification and lineage navigation. This reduces the need for custom tools for standard use cases and accelerates the adoption of the platform by the team.

2.6.2 GraphQL for querying

GraphQL provides a typed and entity-oriented API, consistent with the way the UI interacts with the catalog [17]. In Teseo, this characteristic was considered strategic for two reasons: simplicity in expressing navigation queries and natural alignment with exploratory use cases from the UI.

2.6.3 SDK and API for automation

The Python/Java SDKs represent the recommended option for advanced programmatic operations, leaving GraphQL a role mainly oriented to *user-facing* scenarios [13]. Table 2.1 compares the main interfaces and APIs exposed by Teseo, highlighting their advantages and limits with respect to the different use cases (human interaction or automation).

API	Definition	Advantages	Limitations
Python SDK	SDK	Very flexible and bulk-oriented	Requires understanding of the metadata change events
Java SDK	SDK	Very flexible and bulk-oriented	Requires understanding of the metadata change events
GraphQL API	GraphQL	Intuitive and UI-oriented	Less flexible and requires knowledge of GraphQL
OpenAPI	Low-level	Maximum power and flexibility	Complex and without a dedicated SDK

Table 2.1: Comparison of DataHub interfaces and APIs

2.7 Security and observability

Security was treated as a cross-cutting architectural requirement (indeed, as said previously, the DataHub environment was configured with authentication active to prevent anonymous access to the metadata service and to allow a controlled management of the integration tokens).

The main lines adopted are:

- separation between Teseo/DataHub credentials and the credentials of the ingested source;

- limitation of privileges to only the operations necessary for the ingestion (*least privilege*);
- traceability of the operations through logs and identifiable runs;
- separation of roles between platform administration, integration management and consultation.

Manage Access Tokens

Manage Access Tokens for use with DataHub APIs.

Personal Access Tokens

Personal Access Tokens allow you to make programmatic requests to DataHub's APIs. They inherit your privileges and have a finite lifespan. Do not share Personal Access Tokens.

+ Generate new token

Name	Description	Expires At
------	-------------	------------



No Access Tokens!

Figure 2.6: Teseo section for token management

Figure 2.6 shows the section of the Teseo interface dedicated to the generation and management of the *personal access tokens*, used to authenticate the external integrations towards the metadata service while maintaining separation and revocability of the credentials. From an operational point of view, the combination of application logging, control of the ingestion runs and post-run validation in the UI constitutes the main observability mechanism of the system.

2.8 Metadata lifecycle in the Teseo architecture

To fully understand the architectural design, one must see Teseo as a system governed by the continuous lifecycle of the metadatum, rather than as a static set of components. Indeed, each piece of information goes through distinct phases: acquisition, normalization, persistence, indexing, consultation and update.

In the **acquisition** phase, the connectors extract technical metadata from the sources (structures, dependencies, operational attributes...), minimizing non-deterministic transformations. The quality of this phase is critical: initial mapping errors tend to propagate downstream and to compromise the overall readability of the catalog.

The next phase is **normalization**, in which the extracted data are mapped onto the metadata model. In Teseo, this step was treated as an architectural control point: naming conventions, coherence of the identifiers and granularity of the entities were kept uniform across different sources to avoid semantic duplications.

Then follows the **persistence** phase, in which the normalized information is recorded as aspects of uniquely identifiable entities. In this phase the cardinal principle is idempotence: re-executing an ingestion must not produce structural divergences, but update the state in a predictable way.

Indexing represents the phase that makes the metadata usable on a large scale: search, filters, navigation and access to the most relevant nodes depend on the quality of the indexes.

In the **consultation** phase, technical users and stakeholders query the catalog for different use cases: data discovery, impact analysis, ownership verification and lineage navigation. The architecture must therefore ensure adequate response times and a comprehensible representation of the dependencies.

Finally, the phase of continuous **update** closes and reopens the cycle: changes in the sources require periodic or event-driven refreshes of the catalog, preserving the coherence between the real state of the systems and the state represented in the platform.

2.9 Risk scenarios and mitigation strategies

During the design, some risks typical of centralized metadata platforms were identified. The mitigation of these risks was incorporated into the architectural design.

2.9.1 Risk of lineage incompleteness

The first risk is obtaining an apparently rich catalog but with incomplete lineage on the most critical stretches: this happens when some sources provide limited metadata or when the available permissions do not allow the extraction of some advanced relationships.

The mitigation adopted was twofold, namely making explicit the level of coverage for each integration and clearly separating "core" metadata from "extended" metadata: in this way, the platform maintains a reliable ingestion configuration (*core*), making visible the areas in which the coverage is partial.

2.9.2 Risk of semantic drift

As the number of sources increases, so does the probability of semantic inconsistencies (discordant names, divergent descriptions, duplicated ownership), increasing the complexity of management and reducing the effectiveness of the catalog, even in the presence of technically correct ingestion.

To reduce this problem, it was decided to use common conventions and periodic checks on the quality of the metadata: indeed normalization was not treated as a *one-off*, but as a continuous practice.

2.9.3 Risk of regression

In on-prem environments, updating the components can introduce functional regressions or indirect incompatibilities: for this reason, updates were treated as architectural changes to be validated on a controlled path, including verification of ingestion, search and lineage, before consolidation.

2.10 Observed architectural benefits

The validation carried out over the course of the project highlighted concrete benefits directly attributable to the architectural choices.

- **Reduction of orientation time:** centralized access to the metadata reduces the time needed to understand the origin and use of the assets;
- **Greater predictability of changes:** the availability of the lineage improves the ability to estimate impacts before interventions on sources or transformations;
- **Better collaboration between roles:** data engineers, analysts and governance figures share a common informational base;
- **Organizational scalability:** the onboarding of new sources follows already consolidated patterns, with less dependence on individual knowledge.

In summary, the value of the architecture is not limited to technological correctness, but is reflected in the organization's ability to make faster and more reliable decisions in the presence of complex data ecosystems.

2.11 Positioning of the evolutionary modules

While keeping the focus of the chapter on the core architecture, it is useful to clarify the positioning of the evolutionary modules already envisaged in the overall design. In particular, the advanced governance functionalities and the AI integrations through the MCP protocol were treated as layers overlying the DataHub core, not as substitutes for the metadata catalog (they are additional tools, not central components).

This choice preserves a fundamental principle: the catalog remains the authoritative source of the metadata, while the evolutionary modules act as channels of fruition, automation or decision support.

Chapter 3

Multi-source ingestion and end-to-end lineage

This chapter describes the implementation of the multi-source ingestion flows realized in the Teseo project, with a focus on three main integrations: **Snowflake**, **dbt** and **Power BI**. The technical objective is to build a metadata graph capable of representing the complete dependency between sources, transformations and analytical consumption level, down to the end-to-end **column-level lineage**.

The adopted approach was progressive and repeatable: first the consolidation of the data domain on Snowflake with DDLs structured by layer, then the activation of the transformational lineage through dbt, subsequently the publication of the consumption level with a local *mocked* Power BI semantic model and finally the integration with a Power BI production environment. The result is an end-to-end chain observable in the metadata catalog.

3.1 Snowflake: DDL and setup

Snowflake was used as the cloud data platform to consolidate the domain layers and make the physical dependencies among objects explicit. The separation into databases and schemas (RL/QL/BL + framework metadata) provided a clear perimeter for ingestion and technical governance [3].

In the project, the structure was organized over four main databases:

- DB_DEMO_RL (Raw Layer, i.e. the level at which the data are loaded in raw form, without significant transformations or cleaning);
- DB_DEMO_QL (Quality Layer, i.e. the level at which quality, cleaning and validation rules are applied to the data coming from the Raw Layer);

- DB_DEMO_BL (Business Layer, i.e. the level at which business logics and aggregations are applied to the previously cleaned and validated data);
- DB_DEMO_FMK (Framework Metadata, i.e. the layer of the metadata and of the framework procedures, whose purpose is to provide a coherent and organized structure for the subsequent ingestion steps).

3.1.1 Initial setup and permission validation

Before the complete operational deployment I used some SQL scripts for bootstrapping and verifying the permissions, in particular:

```

1 show grants to user MDOMIZIO;
2 show grants to role TESEO_INTEGRATION_ROLE;
3 show grants on warehouse COMPUTE_WH;
4 show grants on database SNOWFLAKE;

```

Listing 3.1: Initial grants verification

```

1 use role TESEO_INTEGRATION_ROLE;
2 use warehouse COMPUTE_WH;
3
4 create database if not exists DB_DEMO_RL;
5 create database if not exists DB_DEMO_QL;
6 create database if not exists DB_DEMO_BL;
7 create database if not exists DB_DEMO_FMK;
8
9 show databases like 'DB_DEMO_%';
10
11 create schema if not exists DB_DEMO_RL.RL_TMP_PROBE;
12 create schema if not exists DB_DEMO_QL.QL_TMP_PROBE;
13 create schema if not exists DB_DEMO_BL.BL_TMP_PROBE;
14 create schema if not exists DB_DEMO_FMK.FMK_TMP_PROBE;

```

Listing 3.2: Database and schema bootstrap

```

1 show grants to user MDOMIZIO;
2 show grants to role TESEO_INTEGRATION_ROLE;
3 use role TESEO_INTEGRATION_ROLE;
4 use warehouse COMPUTE_WH;
5
6 show databases like 'DB_DEMO_%';
7 show schemas in database DB_DEMO_RL;
8 show schemas in database DB_DEMO_QL;
9 show schemas in database DB_DEMO_BL;
10 show schemas in database DB_DEMO_FMK;
11
12 select count(*) as account_usage_access_ok

```

```

13 from SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
14 where start_time >= dateadd(day, -1, current_timestamp());
15
16 select count(*) as info_schema_access_ok
17 from DB_DEMO_QL.INFORMATION_SCHEMA.COLUMNS;
18
19 select count(*) as process_table_rows
20 from DB_DEMO_FMK.META.MDT_PROCESS_TABLE;
21
22 create schema if not exists DB_DEMO_RL.RL_TMP_TEST;
23 create or replace table DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE (ID number
    , TS timestamp_ntz);
24 insert into DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE values (1,
    current_timestamp());
25 select * from DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE;
26 drop table if exists DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE;

```

Listing 3.3: Permission validation and runtime smoke test

The checks include the visibility of the objects, the runtime context (user, role and warehouse) and the access to `SNOWFLAKE.ACCOUNT_USAGE`, necessary for advanced lineage and usage on the connector side.

3.1.2 DDL deployment of the domain and of the procedures

After performing the bootstrap and the initial validations, the data model was created with a DDL (Data Definition Language) package divided into five thematic files:

- `DDL-01_FOUNDATION.sql`: setup of databases, schemas and framework meta-data tables;
- `DDL-02_MDM_LAYERS.sql`: MDM dimensions (Master Data Management, i.e. management of registry and master data, for example store, product, loyalty card...) on RL, QL and BL;
- `DDL-03_TDM_FACT_LAYERS.sql`: TDM facts (Transactional Data Management, i.e. management of transactional data, for example receipt line, footfall, aggregated KPIs...) on RL, QL and BL;
- `DDL-04_DATAMART_VIEWS.sql`: aggregate views including `VW_DM_TDM_TIMEBAND` and `VW_DM_TDM_DAILY`;
- `DDL-05_PROCEDURES.sql`: loading stored procedures `SP_RL_LOADING`, `SP_QL_LOADING` and `SP_BL_LOADING`.

This structure makes a multilevel lineage observable (external stage, raw, quality, business, datamart) and provides a coherent physical base for the ingestion in DataHub.

From a structural point of view, the pattern applied to the core entities (such as TB_RECEIPT_LINE, TB_STORE and TB_PRODUCT) is coherent across all layers:

- **RL**: raw table with legacy naming and permissive typing (extended VARCHARs), in order to maximize the coverage of the source data;
- **QL**: stream on RL, normalization view (rename + quality rules), typed table with primary key and foreign key constraints, and discard table;
- **BL**: stream on QL, business enrichment view, consolidated table and discard table.

To avoid a description that is too abstract, below I report some representative DDL excerpts of various operations carried out (foundation, raw, typed/quality, business enrichment, datamart and framework procedure).

```

1 CREATE DATABASE IF NOT EXISTS DB_DEMO_RL;
2 CREATE DATABASE IF NOT EXISTS DB_DEMO_QL;
3 CREATE DATABASE IF NOT EXISTS DB_DEMO_BL;
4 CREATE DATABASE IF NOT EXISTS DB_DEMO_FMK;
5
6 CREATE SCHEMA IF NOT EXISTS DB_DEMO_RL.RL_TDM;
7 CREATE SCHEMA IF NOT EXISTS DB_DEMO_RL.RL_MDM;
8 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_TDM;
9 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_TDM_DSC;
10 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_MDM;
11 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_MDM_DSC;
12 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_TDM;
13 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_TDM_DSC;
14 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_MDM;
15 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_MDM_DSC;
```

Listing 3.4: Technical foundation (databases and schemas)

Complete example flow on TB_RECEIPT_LINE:

```

1 CREATE OR REPLACE TABLE DB_DEMO_RL.RL_TDM.TB_RECEIPT_LINE (
2   TS_INGESTION TIMESTAMP_NTZ(9),
3   DTVN VARCHAR, NEGO VARCHAR, NSCN VARCHAR, NRIG VARCHAR,
4   BARC VARCHAR, PZZI VARCHAR, PRZU VARCHAR, ISCN VARCHAR, INTT
   VARCHAR,
5   PIVA VARCHAR, ORVN VARCHAR, NCAS VARCHAR, COPR VARCHAR,
6   FRES VARCHAR, FPRM VARCHAR, CPRM VARCHAR, MPAG VARCHAR, CFID
   VARCHAR,
```

```

7  DIVI VARCHAR, CAMB VARCHAR, STAG VARCHAR, COLL VARCHAR,
8  MARC VARCHAR, REPA VARCHAR, CATG VARCHAR, DCAR VARCHAR
9  );

```

Listing 3.5: Raw layer (legacy source structure)

```

1  CREATE OR REPLACE STREAM DB_DEMO_QL.QL_TDM.ST_QL_TB_RECEIPT_LINE
2  ON TABLE DB_DEMO_RL.RL_TDM.TB_RECEIPT_LINE;
3
4  CREATE OR REPLACE VIEW DB_DEMO_QL.QL_TDM.TVW_TB_RECEIPT_LINE AS
5  SELECT
6    DTVN AS SALE_DATE,
7    NEGO AS STORE_CODE,
8    NSCN AS RECEIPT_NUM,
9    NRIG AS LINE_NUM,
10   BARC AS ITEM_BARCODE,
11   LEFT(ORVN, 2) AS SALE_HOUR,
12   CASE WHEN UPPER(FRES) IN ('Y','S','1') THEN 'Y' ELSE 'N' END AS
    FL_RETURN,
13   CASE WHEN UPPER(FPRM) IN ('Y','S','1') THEN 'Y' ELSE 'N' END AS
    FL_PROMO,
14   CASE
15     WHEN REPLACE(CFID, ' ', '') IN ('O','OO','NA','NULL','') THEN
    NULL
16     ELSE REPLACE(CFID, ' ', '')
17   END AS LOYALTY_CARD_ID
18 FROM DB_DEMO_QL.QL_TDM.ST_QL_TB_RECEIPT_LINE
19 WHERE METADATA\$_ACTION = 'INSERT';
20
21 CREATE OR REPLACE TABLE DB_DEMO_QL.QL_TDM.TB_RECEIPT_LINE (
22   SALE_DATE DATE NOT NULL,
23   STORE_CODE VARCHAR(10) NOT NULL,
24   RECEIPT_NUM VARCHAR(20) NOT NULL,
25   LINE_NUM NUMBER(6,0) NOT NULL,
26   ITEM_BARCODE VARCHAR(14),
27   NET_AMOUNT NUMBER(12,2),
28   CONSTRAINT TB_RECEIPT_LINE_PK PRIMARY KEY (SALE_DATE, STORE_CODE
    , RECEIPT_NUM, LINE_NUM)
29 );

```

Listing 3.6: Quality layer (stream + normalization + typed table)

```

1  CREATE OR REPLACE STREAM DB_DEMO_BL.BL_TDM.ST_BL_TB_RECEIPT_LINE
2  ON TABLE DB_DEMO_QL.QL_TDM.TB_RECEIPT_LINE;
3
4  CREATE OR REPLACE VIEW DB_DEMO_BL.BL_TDM.TVW_TB_RECEIPT_LINE AS
5  SELECT

```



```

6  CURRENT_TIMESTAMP() AS TS_INGESTION,
7  R.SALE_DATE,
8  R.STORE_CODE,
9  R.RECEIPT_NUM,
10 R.LINE_NUM,
11 ROUND(R.NET_AMOUNT * COALESCE(R.EXCHANGE_RATE, 1), 2) AS
    NET_AMOUNT_EUR,
12 CASE
13     WHEN R.SALE_HOUR BETWEEN 0 AND 11 THEN 'MORNING'
14     WHEN R.SALE_HOUR BETWEEN 12 AND 17 THEN 'AFTERNOON'
15     WHEN R.SALE_HOUR BETWEEN 18 AND 23 THEN 'EVENING'
16     ELSE 'UNKNOWN'
17 END AS TIME_BAND,
18 L.CUSTOMER_ID
19 FROM DB_DEMO_BL.BL_TDM.ST_BL_TB_RECEIPT_LINE R
20 LEFT JOIN DB_DEMO_BL.BL_MDM.TB_LOYALTY_CARD L
21     ON R.LOYALTY_CARD_ID = L.LOYALTY_CARD_ID
22 WHERE METADATA\${ACTION} = 'INSERT';

```

Listing 3.7: Business layer (enrichment and consolidation)

```

1  CREATE OR REPLACE VIEW DB_DEMO_BL.BL_TDM.VW_DM_TDM_TIMEBAND AS
2  SELECT
3      R.SALE_DATE,
4      R.STORE_CODE,
5      R.TIME_BAND,
6      COUNT(DISTINCT R.RECEIPT_NUM) AS NUM_RECEIPTS,
7      SUM(R.NET_AMOUNT_EUR) AS NET_REVENUE_EUR,
8      F.TOTAL_ENTRIES,
9      CASE
10         WHEN F.TOTAL_ENTRIES > 0
11         THEN ROUND(COUNT(DISTINCT R.RECEIPT_NUM)::FLOAT / F.
            TOTAL_ENTRIES * 100, 2)
12         ELSE NULL
13     END AS CONVERSION_RATE
14 FROM DB_DEMO_BL.BL_TDM.TB_RECEIPT_LINE R
15 LEFT JOIN (
16     SELECT EVENT_DATE, STORE_CODE, TIME_BAND,
17            SUM(ENTRIES_COUNT) AS TOTAL_ENTRIES
18     FROM DB_DEMO_BL.BL_TDM.TB_FOOTFALL
19     GROUP BY EVENT_DATE, STORE_CODE, TIME_BAND
20 ) F
21     ON R.SALE_DATE = F.EVENT_DATE
22     AND R.STORE_CODE = F.STORE_CODE
23     AND R.TIME_BAND = F.TIME_BAND
24 GROUP BY R.SALE_DATE, R.STORE_CODE, R.TIME_BAND, F.TOTAL_ENTRIES;

```

Listing 3.8: Datamart (aggregation on business view)

Observable logical flow in order:

1. RL_TDM.TB_RECEIPT_LINE
2. QL_TDM.TVW_TB_RECEIPT_LINE / TB_RECEIPT_LINE
3. BL_TDM.TVW_TB_RECEIPT_LINE / TB_RECEIPT_LINE
4. BL_TDM.VW_DM_TDM_TIMEBAND / VW_DM_TDM_DAILY

Finally, as a last step, the framework automates the loading of the layers with dedicated stored procedures. In the QL case (the one used in the complete flow on TB_RECEIPT_LINE) the procedure does not merely call a view, but reads the configuration from the framework metadata and dynamically decides the loading method:

```

1 CREATE OR REPLACE PROCEDURE DB_DEMO_FMK.QL_FMK.SP_QL_LOADING (
2   "TABLE_FULL_NAME" VARCHAR,
3   "CALLER"           VARCHAR DEFAULT 'manual'
4 )
5 RETURNS VARCHAR
6 LANGUAGE PYTHON
7 RUNTIME_VERSION = '3.11'
8 PACKAGES = ('snowflake-snowpark-python')
9 HANDLER = 'main'
10 EXECUTE AS CALLER
11 AS '
12
13 PROCESS_TABLE = "DB_DEMO_FMK.META.MDT_PROCESS_TABLE"
14
15 def get_process_method(session, table_full_name):
16     db, sch, tbl = table_full_name.split(".")
17     proc = session.table(PROCESS_TABLE)
18     row = proc.filter(
19         (col("GN_TARGET_TABLE_DATABASE_NAME") == db) &
20         (col("GN_TARGET_TABLE_SCHEMA_NAME") == sch) &
21         (col("GN_TARGET_TABLE_NAME") == tbl)
22     ).select("GN_PROCESS_METHOD").collect()
23     return row[0][0] if row else "APPEND"
24
25 def load_data(session, table_full_name, source_tbl, pk_names,
26               method):
27     target_cols = session.table(table_full_name).columns
28     cols_str = ", ".join(target_cols)
29
30     if method == "APPEND":
31         session.sql(f"INSERT INTO {table_full_name} ({cols_str})
32         SELECT {cols_str} FROM {source_tbl}").collect()
33     elif method == "REPLACE":

```

```

32     session.sql(f"TRUNCATE TABLE {table_full_name}").collect()
33     session.sql(f"INSERT INTO {table_full_name} ({cols_str})
34     SELECT {cols_str} FROM {source_tbl}").collect()
35 elif method == "MERGE" and pk_names:
36     on_clause = " AND ".join([f"T.{k} = S.{k}" for k in pk_names])
37     merge_sql = f"""
38     MERGE INTO {table_full_name} T
39     USING {source_tbl} S ON {on_clause}
40     WHEN MATCHED THEN UPDATE SET ...
41     WHEN NOT MATCHED THEN INSERT (...)
42     """
43     session.sql(merge_sql).collect()
44 # ... python body ...
45 ';

```

Listing 3.9: Stored procedure syntax and load strategies

The main of the same procedure explicitly shows the complete technical flow of the quality control before the actual final load:

```

1 def main(session, table_full_name, caller):
2     db, sch, tbl = table_full_name.split(".")
3     dsc_tbl = f"{db}.{sch}_DSC.DSC_{tbl}"
4     tvw = f"{db}.{sch}.TVW_{tbl}"
5     batch_tbl = f"{db}.{sch}.BATCH_{tbl}"
6     cast_tbl = f"{batch_tbl}_CAST"
7
8     session.sql(f"CREATE OR REPLACE TEMPORARY TABLE {batch_tbl} LIKE
9     {tvw}").collect()
10    session.sql(f"CREATE OR REPLACE TEMPORARY TABLE {cast_tbl} LIKE
11    {table_full_name}").collect()
12    session.sql(f"INSERT INTO {batch_tbl} SELECT * FROM {tvw}").
13    collect()
14
15    pk_names = get_pk_names(session, table_full_name)
16    method = get_process_method(session, table_full_name)
17
18    pk_err = check_pk_duplicates(session, batch_tbl, dsc_tbl,
19    pk_names)
20    nn_err = check_not_null(session, table_full_name, batch_tbl,
21    dsc_tbl)
22    dt_err = check_data_types(session, table_full_name, batch_tbl,
23    cast_tbl, dsc_tbl)
24
25    load_log = load_data(session, table_full_name, cast_tbl,
26    pk_names, method)
27
28    session.sql(f"DROP TABLE IF EXISTS {batch_tbl}").collect()

```

```
22 session.sql(f"DROP TABLE IF EXISTS {cast_tbl}").collect()
```

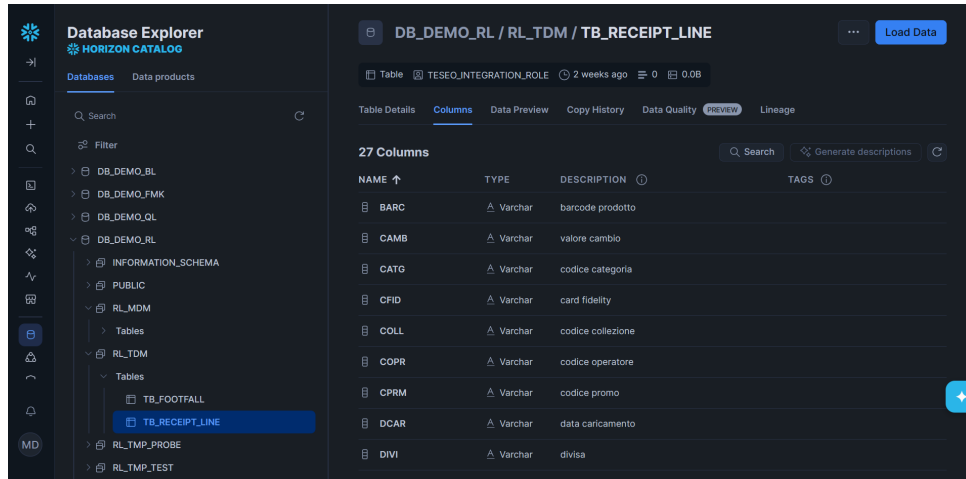
Listing 3.10: DQ orchestration and final load

For the case study reported in this chapter (receipt line), the procedure file also contains the actual calls of the three steps $RL \rightarrow QL \rightarrow BL$:

```
1 CALL DB_DEMO_FMK.RL_FMK.SP_RL_LOADING(
2   'DB_DEMO_RL.RL_TDM.TB_RECEIPT_LINE',
3   'DB_DEMO_RL.RL_TDM.STG_TB_RECEIPT_LINE',
4   'DB_DEMO_RL.RL_TDM.FF_CSV',
5   'CSV',
6   'manual'
7 );
8
9 CALL DB_DEMO_FMK.QL_FMK.SP_QL_LOADING(
10  'DB_DEMO_QL.QL_TDM.TB_RECEIPT_LINE',
11  'manual'
12 );
13
14 CALL DB_DEMO_FMK.BL_FMK.SP_BL_LOADING(
15  'DB_DEMO_BL.BL_TDM.TB_RECEIPT_LINE',
16  'manual'
17 );
```

Listing 3.11: Actual calls of the TB_RECEIPT_LINE flow

In the end, after loading the entire domain and the procedures, from the Snowflake UI it is possible to observe the structure of the databases and their schemas, reported in Figure 3.1:

**Figure 3.1:** Database and schema structure in Snowflake

3.2 DataHub configuration

The platform was started with DataHub Core v1.4.0 in a local Docker environment, with metadata service authentication enabled both on the GMS service and on the frontend:

```

1 datahub-gms-quickstart:
2   environment:
3     METADATA_SERVICE_AUTH_ENABLED: 'true'
4
5 frontend-quickstart:
6   environment:
7     METADATA_SERVICE_AUTH_ENABLED: 'true'

```

Listing 3.12: DataHub Docker Compose excerpt (auth enabled)

The complete ingestion process was executed in a Python virtual environment in order to obtain an isolated and repeatable behavior, with the dependencies necessary for the Snowflake and dbt connectors, but without compatibility or version errors. Below are the main commands to bootstrap the environment and start the quickstart (which uses the local customized docker-compose with authN enabled):

```

1 python -m venv venv
2 venv\Scripts\Activate
3 python -m pip install --upgrade pip wheel setuptools
4 pip install acryl-datahub dbt-core dbt-snowflake
5 pip install "acryl-datahub[snowflake]"
6
7 datahub docker quickstart --quickstart-compose-file .\docker-
   compose.yml --version v1.4.0
8 datahub init

```

Listing 3.13: Main bootstrap commands

3.3 Snowflake ingestion

Now that both Snowflake and DataHub are configured, I moved on to configuring the ingestion by means of `recipe.yml`. In particular, I decided to limit the configuration to the perimeter of the demo databases and of the functional schemas used, including both the tables and the views and activating the possibility of extracting the lineage at the column level.

As for the access **credentials** (and secrets in general), I adopted a more secure strategy of not hardcoding anything in the recipe, but of using **environment**

variables that are loaded from a dedicated `.env` file. In this way, the recipe is completely agnostic with respect to the environment and can be easily reused or shared without risks of credential exposure.

The **complete recipe** to extract the metadata only from Snowflake is the following:

```
1 pipeline_name: snowflake_prod_main
2
3 source:
4   type: snowflake
5   config:
6     username: "${SNOWFLAKE_USERNAME}"
7     password: "${SNOWFLAKE_PASSWORD}"
8     account_id: "${SNOWFLAKE_ACCOUNT_ID}"
9     role: "${SNOWFLAKE_ROLE}"
10    warehouse: "${SNOWFLAKE_WAREHOUSE}"
11    platform_instance: "${SNOWFLAKE_PLATFORM_INSTANCE}"
12
13    database_pattern:
14      allow:
15        - DB_DEMO_RL
16        - DB_DEMO_QL
17        - DB_DEMO_BL
18        - DB_DEMO_FMK
19
20    schema_pattern:
21      allow:
22        - DBT
23        - RL_TDM
24        - RL_MDM
25        - QL_TDM
26        - QL_TDM_DSC
27        - QL_MDM
28        - QL_MDM_DSC
29        - BL_TDM
30        - BL_TDM_DSC
31        - BL_MDM
32        - BL_MDM_DSC
33        - META
34        - RL_FMK
35        - QL_FMK
36        - BL_FMK
37
38    include_tables: true
39    include_views: true
40    include_column_lineage: true
41    include_usage_stats: false
42
```

```
43     profiling:
44         enabled: false
45     stateful_ingestion:
46         enabled: false
47
48 sink:
49     type: datahub-rest
50     config:
51         server: "${DATAHUB_GMS_URL}"
52         token: "${DATAHUB_TOKEN}"
53         timeout_sec: 120
54         retry_max_times: 5
55         max_threads: 1
```

Listing 3.14: Snowflake recipe

Below I report the operational meaning of some of the options that played a fundamental role within my recipe, in particular as regards the quality of the lineage graph and the stability of the ingestion:

- `include_column_lineage:true`: enables the lineage with column-level granularity when the SQL parser manages to reconstruct the dependencies; it is the key option for moving from a table-only lineage to a lineage useful for impact analysis;
- `include_usage_stats:false`: disables the collection of usage statistics (query history/usage) during the ingestion. In my initial setup I favored stability and perimeter, deferring the enrichment with usage to a later phase;
- `profiling.enabled:false`: avoids the profiling of the data (cardinality, *null ratio*...), thus reducing time, computational cost and required permissions (as said above, I gave priority to stability and perimeter);
- `stateful_ingestion.enabled:false`: avoids the soft-delete mechanism for objects no longer found by the source. I kept this option disabled in the main recipe so as not to risk unwanted removals during the first exploratory runs;
- `database_pattern.allow` and `schema_pattern.allow`: restrict the perimeter to only the project databases and schemas, reducing the noise in the catalog and the scanning times.

The execution can occur through single commands, but I decided to also write a **PowerShell wrapper** `ingest.ps1` that automatically loads the variables from `.env` and launches the ingestion command. This choice was made to simplify recurring execution, reduce the risk of typing errors at each command and make

the orchestration more **modular**, so as to be able to easily extend the wrapper to also include, in the next steps, the dbt and Power BI ingestion:

```

1 param(
2   [switch]\$IncludeDbt,
3   [switch]\$IncludePowerBIMock,
4   [switch]\$SkipSnowflake,
5   [switch]\$ReconcileDbtSnowflakeBridge,
6   [string]\$SnowflakeRecipe = 'venv\Ingestion\recipe_snowflake.yml',
7   [string]\$DbtRecipe = 'venv\Ingestion\recipe_dbt.yml',
8   [string]\$SnowflakeBridgeRecipe = 'venv\Ingestion\
9   recipe_snowflake_dbt_bridge.yml'
10 )
11 # Load .env into process scope
12 Get-Content \$envFile | Where-Object { \$_ -match '=' } | ForEach-
13   Object {
14     \$name, \$value = \$_ -split '=', 2
15     [Environment]::SetEnvironmentVariable(\$name, \$value, 'Process')
16   }
17 datahub ingest -c \$snowflakeRecipePath
18 if (\$IncludeDbt) { datahub ingest -c \$dbtRecipePath }
19 if (\$ReconcileDbtSnowflakeBridge) { datahub ingest -c \
    \$bridgeRecipePath }

```

Listing 3.15: Excerpt of the ingest.ps1 orchestrator

3.4 dbt ingestion and connection with Snowflake

After the publication of the Snowflake perimeter, I introduced **dbt** to make the logical transformation explicit (semantic model) and obtain a more readable lineage between sources, intermediate layers and final marts. Operationally, the work was divided into two main strands: building reliable dbt artifacts and configuring the DataHub recipe (with anti-noise filters as seen previously in the Snowflake recipe).

3.4.1 dbt project setup and artifact generation

The dbt project was organized from the very beginning by clearly separating:

- **full_coverage**: auto-generated models in order to extensively cover the Snowflake objects;
- **semantic layer**: curated models for the analytical flow (`stg` → `int` → `mart`).

The choice to materialize **stg** and **int** as views and **mart** as tables was made to find a trade-off between traceability and performance. The dbt profile points to Snowflake and publishes the results in the DBT schema.

The declared sources instead cover RL/QL/BL, DSC (discard) areas and meta-framework; in this way the dbt models can reference both operational data and quality control tables, maintaining lineage continuity.

The dbt pipeline used to produce artifacts coherent with the DataHub ingestion is the following:

```
1 dbt clean
2 dbt deps
3 dbt build --no-partial-parse
4 dbt docs generate --no-partial-parse
```

Listing 3.16: dbt build and artifacts for ingestion

The artifacts **manifest.json**, **catalog.json** and **run_results.json** are then referenced by the dbt recipe.

In the command flow shown above, **dbt build --no-partial-parse** and **dbt docs generate --no-partial-parse** were executed intentionally to avoid cache side effects during the evolution of the project: in this way the artifacts used by DataHub are always aligned with the real state of the code.

3.4.2 dbt recipe and noise control in the graph

The **recipe_dbt.yml** is configured to:

- target platform **snowflake**;
- column-level lineage (**include_column_lineage:true**);
- dbt/Snowflake URN alignment (platform instance and lowercase conversion);
- selective inclusion of only the semantic models **stg_**, **int_**, **mart_** to reduce non-useful entities (*noise*).

This configuration therefore solves three problems typical of the dbt ingestion:

- **URN alignment:** with **target_platform_instance** aligned to Snowflake, the dbt nodes are correctly attached to the physical objects already ingested, avoiding duplications and disconnections in the graph;
- **noise reduction:** the **node_name_pattern** filter limits the graph to the semantically relevant models, avoiding saturation with technical entities useless for the purpose of the lineage;

- **incremental coherence:** `stateful_ingestion` in the dbt configuration makes it possible to remove stale metadata when names or models change (thus avoiding discrepancies between the metadata).

```

1 pipeline_name: dbt_prod_main
2
3 source:
4   type: dbt
5   config:
6     manifest_path: "${DBT_MANIFEST_PATH}"
7     catalog_path: "${DBT_CATALOG_PATH}"
8     run_results_paths:
9       - "${DBT_RUN_RESULTS_PATH}"
10    target_platform: "snowflake"
11    env: "${DBT_ENVIRONMENT}"
12    platform_instance: "${DBT_PLATFORM_INSTANCE}"
13    target_platform_instance: "${SNOWFLAKE_PLATFORM_INSTANCE}"
14    include_column_lineage: true
15    convert_column_urns_to_lowercase: true
16    infer_dbt_schemas: true
17    include_database_name: true
18    dbt_is_primary_sibling: false
19    node_name_pattern:
20      allow:
21        - "^model\\.teseo_dwh\\.(stg_|int_|mart_).*[\\$]"
22        - "^source\\.teseo_dwh\\.\\.\\.*[\\$]"
23    stateful_ingestion:
24      enabled: true
25      remove_stale_metadata: true
26      fail_safe_threshold: 100.0
27
28 sink:
29   type: datahub-rest
30   config:
31     server: "${DATAHUB_GMS_URL}"
32     token: "${DATAHUB_TOKEN}"
33     timeout_sec: 120
34     retry_max_times: 5
35     max_threads: 1

```

Listing 3.17: dbt recipe

3.4.3 Snowflake-dbt bridge reconciliation

To remove any inconsistencies in the physical schema `DB_DEMO_BL.DBT` alone, I also used a third dedicated recipe `recipe_snowflake_dbt_bridge.yml`, with `stateful_ingestion` enabled and a deliberately restricted scope:

```

1 pipeline_name: snowflake_dbt_bridge_reconcile
2 source:
3   type: snowflake
4   config:
5     ...
6     include_tables: true
7     include_views: true
8     include_column_lineage: true
9     stateful_ingestion:
10      enabled: true
11      remove_stale_metadata: true
12 sink:
13   type: datahub-rest
14   config:
15     server: "${DATAHUB_GMS_URL}"

```

Listing 3.18: Snowflake-dbt bridge recipe

The complete orchestration Snowflake + dbt + bridge is executed with a single invocation (`.\ingest.ps1 -IncludeDbt -ReconcileDbtSnowflakeBridge`). The result of this execution is observable in Figure 3.2: the image shows the column-level lineage obtained after the combined ingestion of Snowflake and dbt, in which each column of the dbt models is linked to the physical Snowflake columns from which it derives, confirming the correct reconciliation of the URNs between the two connectors.

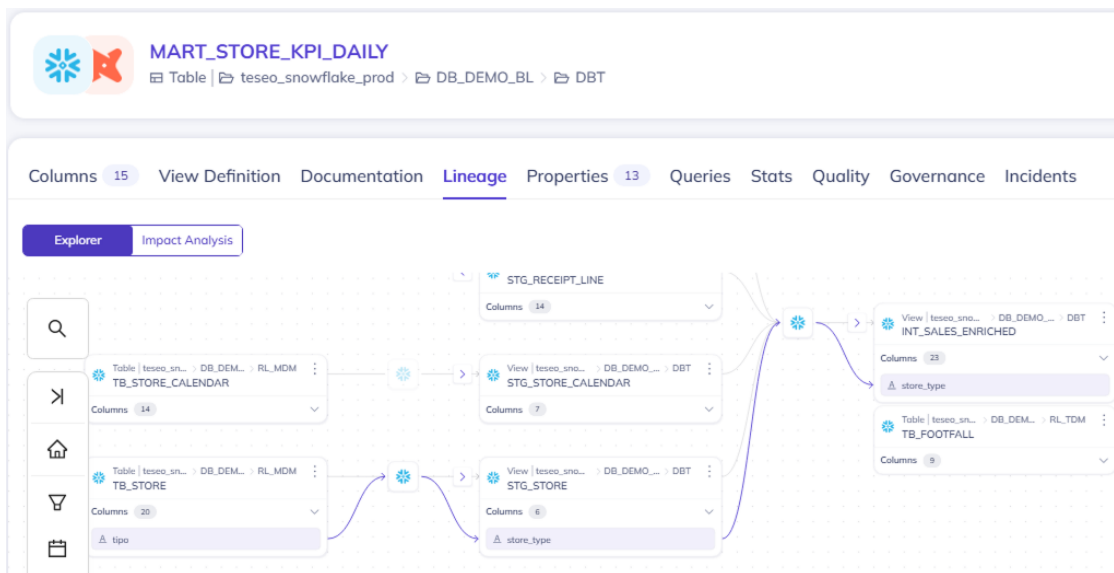


Figure 3.2: Lineage result after Snowflake + dbt ingestion

3.5 Power BI ingestion: from local mock to service

In the Teseo project a Power BI semantic model ready to be used as an ingestion base was not available. For this reason I built a local mock structure with the same informational objective as the real case: to publish on Teseo's GMS the workspace, the semantic model, the reports and the lineage, always at the column/field level.

As always, the approach was designed to be repeatable and verifiable: first the generation of the mock structures, then the creation of the ingestion plan, finally the emission towards DataHub with automatic post-run validation checks.

3.5.1 Motivation for the local model

A legitimate question might be: why build a local mock instead of building the semantic model and the reports directly on a real Power BI Service? The main motivation was technical: without a pre-existing Power BI model, the mock made it possible to quickly build and test the final stretch of the lineage (dbt → BI semantic model) without immediately depending on the tenant, workspace and Service permissions, and therefore without loading test structures into a production environment.

The local mock is mainly composed of two central artifacts:

- `powerbi_mock_entities_dbt.yaml` and the *full* version: they define workspaces, semantic models, tables, columns, reports, pages and visuals;
- `powerbi_lineage_map_dbt.csv` and the *full* version: they define the explicit link between Power BI columns and dbt upstream (with support for `upstream_urn` for the unique reference).

This choice solved two practical needs of my case: avoiding name mismatches due to the manual creation of the structures and immediately obtaining a fine-grained lineage consistent with the dbt models already published previously (without having to modify them).

An excerpt of the mock semantic model shows that the structure is already organized as a real BI model:

```

1 project:
2   name: Teseo Snowflake dbt Power BI Mock Full
3 powerbi:
4   platform: powerbi
5   platform_instance: local_desktop

```

```

6 workspace:
7   id: ws_teseo_uat
8   name: Teseo UAT
9 semantic_models:
10   - name: dbt_full_semantic
11     tables:
12       - name: AUTOGEN__DB_DEMO_BL__BL_MDM__TB_LOYALTY_CARD
13         columns: [TS_INGESTION, LOYALTY_CARD_ID, CUSTOMER_ID,
CARD_TYPE]

```

Listing 3.19: Excerpt of Power BI entities from the dbt catalog

The CSV mapping completes the model with the column-to-column link towards the dbt upstream:

```

1 workspace_id,report_name,pbi_dataset_name,pbi_table_name,
  pbi_column_name,upstream_platform,upstream_dataset,
  upstream_column,upstream_urn
2 ws_teseo_uat,DBT Business Semantic Dashboard,dbt_full_semantic,
  AUTOGEN__DB_DEMO_BL__BL_MDM__TB_LOYALTY_CARD,LOYALTY_CARD_ID,
  dbt,teseo_dbt_semantic.DB_DEMO_BL.DBT.
  autogen__db_demo_bl__bl_mdm__tb_loyalty_card,LOYALTY_CARD_ID,"
  urn:li:dataset:(urn:li:dataPlatform:dbt,teseo_dbt_semantic.
  DB_DEMO_BL.DBT.autogen__db_demo_bl__bl_mdm__tb_loyalty_card,
  PROD)"

```

Listing 3.20: Excerpt of Power BI mock mapping to dbt

3.5.2 Generation and ingestion scripts

The mock pipeline was implemented with Python and PowerShell scripts that clearly separate generation, ingestion and verification. In particular:

- `generate_powerbi_from_dbt.py`: reads the `dbt catalog.json` and automatically generates the YAML entities from the catalog and the related CSV mapping, with naming logic coherent with the real model (using the `AUTOGEN_` prefix to highlight the automatically generated tables);
- `ingest_mock_powerbi.py`: builds `powerbi_mock_plan.json` and, in real mode, emits MCP events towards DataHub containing schema, dataset-level lineage and fine-grained lineage;
- `ingest_mock_powerbi_dbt_full.ps1`: orchestration file that executes the automatic generation from dbt and then the ingestion, with a `-DryRun` option to validate the generation without emission (useful for debugging purposes);
- `ingest_mock_powerbi_bundle.ps1`: combines two runs (full semantic + mart-focused) to cover different reporting scenarios.

```

1 # verify plan without sending
2 .\powerbi_mock\scripts\ingest_mock_powerbi_dbt_full.ps1 -DryRun
3
4 # real ingestion of the full semantic model
5 .\powerbi_mock\scripts\ingest_mock_powerbi_dbt_full.ps1
6
7 # complete bundle: 2 full reports + 1 mart report
8 .\powerbi_mock\scripts\ingest_mock_powerbi_bundle.ps1

```

Listing 3.21: Execution of the Power BI mock pipeline

As anticipated, the ingestion script in particular includes two very useful functions to reduce the real integration errors:

- auto-resolution of the `upstream_urn` against DataHub;
- auto-resolution of the `upstream_column` with respect to the actual `fieldPath` in the metadata schema.

These options make the mock very robust when the naming changes between dbt, Snowflake and PBI.

3.5.3 Connection to Power BI Service

After the validation of the *mock* flow, in the project I also activated the ingestion on a real Power BI Service, maintaining the same setting of operational robustness: credentials external to the recipe, preliminary connectivity test and perimeter restricted to the selected workspace.

The recipe that makes it possible to *ingest* the Power BI layer enables the extraction of reports, dashboards, dataset schema and lineage (also *fine-grained* as far as possible), with mapping of the Snowflake datasource towards the same *platform instance* already used in the Snowflake and dbt perimeter:

```

1 pipeline_name: powerbi_real_main
2
3 source:
4   type: powerbi
5   config:
6     tenant_id: "${POWERBI_TENANT_ID}"
7     client_id: "${POWERBI_CLIENT_ID}"
8     client_secret: "${POWERBI_CLIENT_SECRET}"
9
10    workspace_id_pattern:
11      allow:
12        - "^${POWERBI_WORKSPACE_ID}$"
13
14    workspace_type_filter:

```

```

15     - "Workspace"
16
17     env: "${POWERBI_ENV}"
18     platform_instance: "${POWERBI_PLATFORM_INSTANCE}"
19     extract_reports: true
20     extract_dashboards: true
21     extract_dataset_schema: true
22     extract_lineage: true
23     extract_column_level_lineage: true
24     convert_lineage_urns_to_lowercase: true
25
26     server_to_platform_instance:
27       "${POWERBI_SNOWFLAKE_SERVER}":
28         platform_instance: "${SNOWFLAKE_PLATFORM_INSTANCE}"
29         env: "${POWERBI_ENV}"
30
31   sink:
32     type: datahub-rest
33     config:
34       server: "${DATAHUB_GMS_URL}"
35       token: "${DATAHUB_TOKEN}"

```

Listing 3.22: Power BI Service recipe

This phase in the project proved to be functional and confirmed the possibility of publishing metadata and lineage also from the real Service [5, 6].

Subsequently, in Teseo’s evolutionary path, this mode was progressively flanked by a generative approach: given the good outcome of the Power BI ingestion, it was decided to move to developing a solution in which the Power BI frontend is created by means of the MCP tools (reducing the manual steps and making the alignment between technical metadata and analytical consumption level more coherent) and published as a custom Power BI built *ad hoc* starting from the complete lineage and in particular from the *ingested* level closest to the end user [18, 19].

3.6 Construction of the end-to-end lineage

With the chain Snowflake → dbt → Power BI, the resulting graph shows an end-to-end lineage navigable from the physical layer to the consumption level. This step is what makes Teseo particularly powerful: not only cataloging, but **complete traceability** ready for impact analysis, troubleshooting and automated inspection.

From a practical point of view, end-to-end observability enables three key capabilities:

- **immediate impact analysis:** from a source column it is possible to trace all the downstream objects (up to the semantic models, reports and visuals);
- **coherent semantic governance:** ownership, tags, glossary terms and descriptions remain aligned along the entire transformation chain and can also be propagated or enriched at every point of the flow;
- **acceleration of automations:** thanks to the complete lineage, Teseo's MCP tools can propose or apply targeted updates with already validated context and with a broad scope over the entire metadata graph.

A typical *fine-grained lineage* path observable in the graph is:

1. source table in Snowflake;
2. dbt model with transformation lineage;
3. semantic model, report and Power BI visual with explicit column dependencies.

Figure 3.3 shows an example of this complete chain as it appears in Teseo's graph:

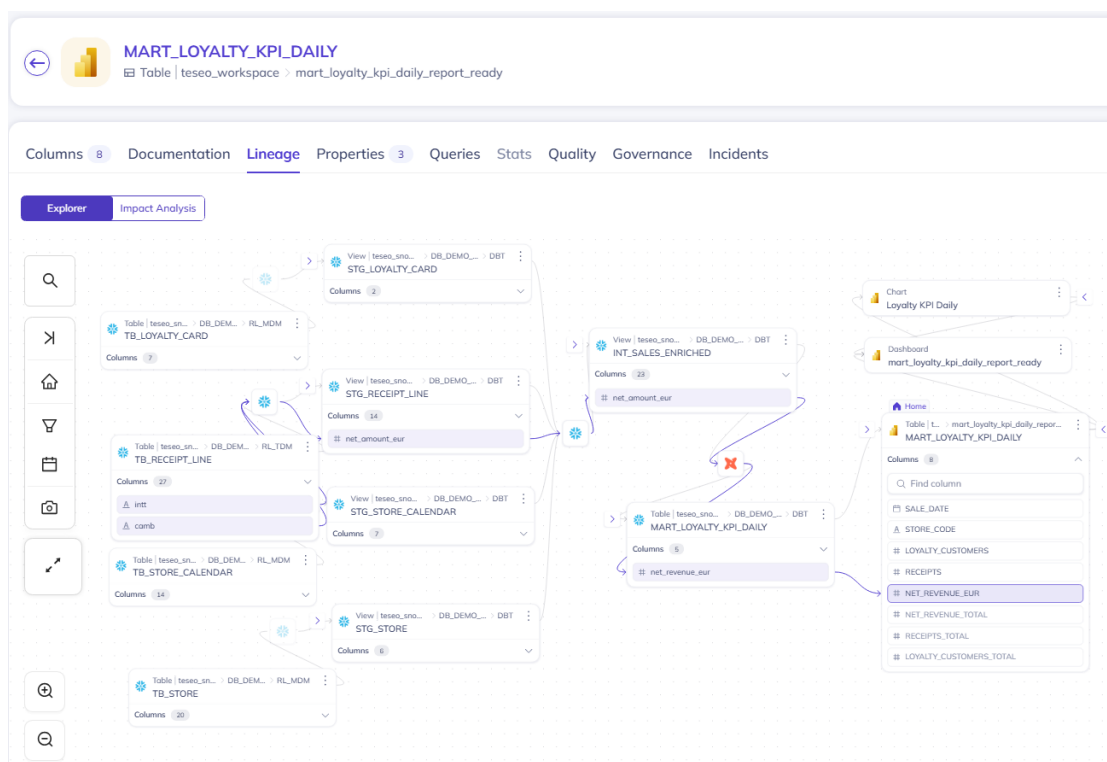


Figure 3.3: End-to-end lineage in Teseo

3.7 Example of ingestion from other sources

The pattern implemented in Teseo (recipes + validations + coherent URN mapping + noise control) is reusable also for other sources, maintaining the same objective: to build a lineage useful for governance and automation, not just a huge technical glossary [5, 9].

3.7.1 Databricks case (Unity Catalog)

Databricks is another excellent example case because it combines metastore, notebooks, job activity and lineage. In the official DataHub documentation, the `databricks` source (*Unity Catalog*) already exposes native capabilities on schema, ownership, usage and lineage (including options for *column-level lineage*) [20].

A minimal recipe for demonstration purposes is the following:

```

1 pipeline_name: databricks_uc_main
2
3 source:
4   type: databricks
5   config:
6     workspace_url: "${DATABRICKS_WORKSPACE_URL}"
7     client_id: "${DATABRICKS_CLIENT_ID}"
8     client_secret: "${DATABRICKS_CLIENT_SECRET}"
9     platform_instance: "${DATABRICKS_PLATFORM_INSTANCE}"
10    env: "${DATABRICKS_ENV}"
11
12    include_tables: true
13    include_views: true
14    include_table_lineage: true
15    include_column_lineage: true
16    include_usage_statistics: true
17    include_ownership: true
18
19    warehouse_id: "${DATABRICKS_WAREHOUSE_ID}"
20
21    catalog_pattern:
22      allow:
23        - "^main$"
24        - "^analytics$"
25
26    stateful_ingestion:
27      enabled: true
28      remove_stale_metadata: true
29      fail_safe_threshold: 100.0
30
31 sink:
```

```
32  type: datahub-rest
33  config:
34    server: "${DATAHUB_GMS_URL}"
35    token: "${DATAHUB_TOKEN}"
```

Listing 3.23: Databricks recipe

In this scenario, the observable lineage is readable as a chain **bronze -> silver -> gold -> BI**: a raw table on the **main** catalog, a curated view or table on **analytics.silver**, an **analytics.gold** table and finally a consumption level (dashboard or semantic model). The practical effect is analogous to what was obtained in Teseo: from a single business column it is possible to trace back to the source and vice versa, with strong support for impact analysis.

3.7.2 The true value of Teseo

This example shows how Teseo is by design extensible: once the recipe suited to the use case and the quality control of the lineage are set, the infrastructure can incorporate new sources with incremental effort and with immediate return in terms of observability and governance.

In other words, the power of a tool like Teseo does not lie only in the single connector used today, but in the ability to **reuse** the same framework on **heterogeneous** domains and platforms while maintaining a unified end-to-end view of the metadata.

Chapter 4

Data Governance

This chapter describes how Teseo operationally enables the **Governance** of the enterprise information assets: indeed, after consolidating multi-source ingestion and lineage, the next step is to make the obtained metadata governable, standardized and verifiable over time.

In the context of the project, governance was not treated as an accessory layer, but as a **structural component** of the platform: asset classification, responsibilities, business taxonomy, quality control and auditability are all aspects that leverage the same metadata graph already built in the previous chapters [9, 14, 17].

4.1 Data Governance and metadata

4.1.1 What Data Governance is

By **Data Governance** we mean the set of principles, roles, processes and controls with which an organization ensures that data are [21, 22]:

- **reliable** (accurate, complete and up to date);
- **understandable** (documented, contextualized and semantically coherent);
- **accountable** (explicit owners, validation processes and accountability);
- **compliant** (aligned with regulatory constraints and internal policies);
- **reusable** (effective discovery and low friction within the team).

In other words, governance is not an isolated functionality, but a continuous system of decision and control over the life cycle of the data.

In the modern context, governance operates above all on the basis of **metadata**: without metadata it is not possible to know who owns an asset, what business meaning it has, how quality evolves over time and which dependencies it produces downstream.

For this reason, Teseo adopts a *metadata-first* strategy: ingestion, lineage and governance share the same model and the same identifier (URN), allowing a unified control over the assets [14].

4.1.2 Main dimensions of governance

Within Teseo's perimeter, governance is implemented on several complementary dimensions:

- **semantics** (*knowledge*): glossary, descriptions and structured properties;
- **organization** (*ownership*): ownership, domains, groups and roles;
- **quality** (*quality*): assertions, monitoring and incidents;
- **security** (*security*): workflows, security and automations (also from API/CLI);
- **compliance and access** (*accessibility*): classifications, policies and approvals.

Figure 4.1 summarizes these five dimensions, reporting for each one the concrete mechanisms with which Teseo implements them.

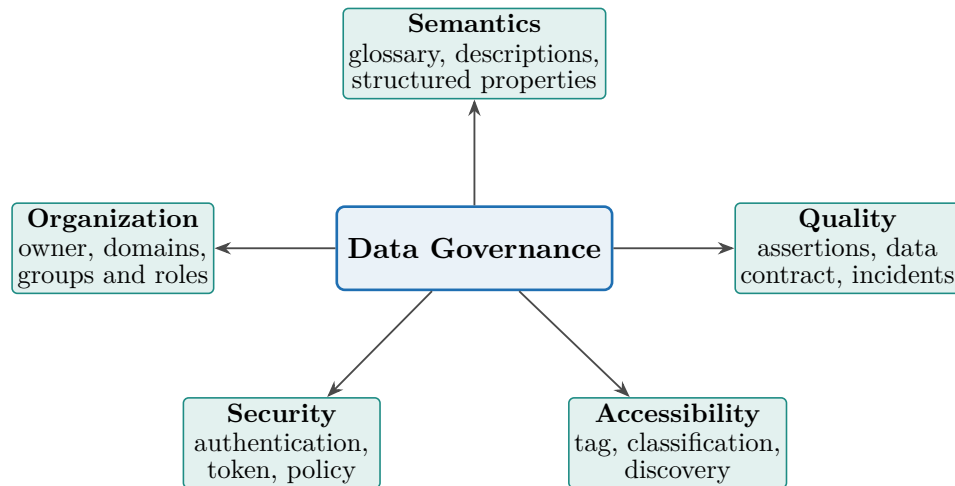


Figure 4.1: Data Governance in Teseo

4.2 Role of governance in Teseo

In Teseo, DataHub Core plays the role of *control plane* of the metadata: each asset (dataset, column, dashboard...) is described by semantically typed aspects and can therefore be subject to common governance rules.

The main benefits of this approach are:

- **standardization:** tags, domains, ownership and glossary follow shared conventions;
- **traceability:** every change to the metadata can be traced back to specific entities and URNs;
- **unification:** governance insists on the same objects already used for ingestion and lineage;
- **scalability:** the processes can be managed from UI, CLI, GraphQL or SDK with a coherent model.

4.3 Governance entities in Teseo

4.3.1 Tagging and classification

Tags make it possible to classify assets and columns quickly and flexibly. For example, they can be used to distinguish sensitive entities (e.g. PII), datasets certified for reporting, technical layers or areas at qualitative risk.

A tag can be added to a dataset through the UI or also through the GraphQL API, for example to indicate that it is deprecated:

```
1 mutation addTags {  
2   addTags(  
3     input: {  
4       tagUrns: ["urn:li:tag:deprecated"],  
5       resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,  
6         fct_users_created,PROD)"  
7     }  
8   )  
}
```

Listing 4.1: Adding tags to a dataset via GraphQL

Similarly, the tag can be removed when it is no longer valid:

```
1 mutation removeTag {
2   removeTag(
3     input: {
4       tagUrn: "urn:li:tag:deprecated",
5       resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
6         fct_users_created,PROD)"
7     }
8   )
9 }
```

Listing 4.2: Removing a tag from a dataset via GraphQL

4.3.2 Domains and ownership

Governance on Teseo also includes the use of **domains**, which make it possible to organize and categorize the assets based on the context they belong to (e.g. Sales, Finance, Marketing...), thus improving their discovery and accountability. In parallel, **ownership** makes the technical and business responsibility over the various assets explicit, so as to facilitate their management and traceability.

As for tags, domains and ownership can also be managed both from the UI and from the API. Below is an example of assigning a domain to a dataset via GraphQL:

```
1 mutation setDomain {
2   setDomain(
3     domainUrn: "urn:li:domain:marketing",
4     entityUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
5       fct_users_created,PROD)"
6   )
7 }
```

Listing 4.3: Assigning a domain to a dataset via GraphQL

Ownership can also be assigned via GraphQL, for example to indicate the technical owner of a dataset:

```
1 mutation addOwner {
2   addOwner(
3     input: {
4       ownerUrn: "urn:li:corpuser:data.engineer",
5       resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
6         fct_users_created,PROD)",
7       ownerEntityType: CORP_USER,
8       type: TECHNICAL_OWNER
9     }
10  }
```

```

9   )
10  }

```

Listing 4.4: Assigning a technical owner via GraphQL

4.3.3 Business Glossary and Structured Properties

To align technical language and business language, there are two components with complementary roles:

- **Glossary Terms:** shared and reusable definitions of *business* concepts;
- **Structured Properties:** typed and validatable attributes to extend the semantics of the assets.

In the operational model, glossaries make it possible to reduce interpretive ambiguities, while structured properties make it possible to create more precise policies (for example criticality, classification, service level...).

Below is an example of adding a glossary term to a dataset via GraphQL:

```

1  mutation addTerms {
2    addTerms(
3      input: {
4        termUrns: ["urn:li:glossaryTerm:finance.revenue"],
5        resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
6          fct_users_created,PROD)"
7      }
8    )
9  }

```

Listing 4.5: Adding a glossary term to a dataset via GraphQL

4.3.4 Assertions and Data Contracts

Governance in Teseo does not concern only the pure cataloging aspect, but also the ability to demonstrate the quality of the metadata catalog. To do this, Teseo supports several types of assertions (i.e. verifiable statements on specific assets):

- **column:** verifies that a column/field respects certain conditions (e.g. not null, cardinality, value distribution);
- **freshness:** verifies that a dataset is updated within an expected time window;
- **volume:** verifies that a dataset has a record volume within certain limits (thus in terms of *size*);

- **schema**: verifies that the schema of a dataset respects an expected definition;
- **custom**: custom assertions based on SQL queries written specifically to verify specific criteria.

In addition to this, Teseo makes it possible to compose **Data Contracts** based on selected assertions, since a data contract is a combination of assertions that define the expectations on an asset (like a *package* of assertions).

These verification structures make it possible to move from purely static documentation to a dynamic **policy + verification** model, in which the expectations on a dataset become executable controls that can be monitored over time by means of monitoring and reporting tools.

Below is an example of a freshness assertion written via GraphQL:

```

1 mutation upsertDatasetFreshnessAssertionMonitor {
2   upsertDatasetFreshnessAssertionMonitor(
3     input: {
4       entityUrn: "urn:li:dataset:(urn:li:dataPlatform:snowflake,
5         analytics.orders,PROD)",
6       evaluationSchedule: {
7         timezone: "Europe/Rome",
8         cron: "0 */8 * * *"
9       },
10      mode: ACTIVE
11    }
12  ) {
13    urn
14  }

```

Listing 4.6: Freshness assertion via GraphQL

4.3.5 Incident management

When an assertion/contract fails or an unexpected anomaly emerges, the platform makes it possible to open and track **incidents** associated with specific assets: an incident is an event that indicates a problem with a specific asset, flagging the latter as non-compliant. In terms of governance, this reduces the mean diagnostic time and improves the communication between data owner, data engineer and business-side stakeholders.

Indeed, incidents are strongly integrated with the *assertions* framework: when a quality control configured to generate incidents fails, the incident is created automatically and maintains the link with the source that generated it (through

`sourceUrn`) so as to ensure the traceability between event, violated rule and involved assets, and to clearly distinguish *automatic* incidents (from monitoring) and *manual* ones (from operational reporting) [23, 24].

The model of an incident typically includes some properties such as [24]:

- type (freshness, volume, schema...);
- priority (CRITICAL, HIGH, MEDIUM, LOW);
- status (ACTIVE, RESOLVED);
- possible assignees and list of impacted entities (dataset, dashboard, chart...).

On the governable assets (i.e. dataset, dashboard, chart, dataFlow, dataJob and schemaField), Teseo also exposes an **incidentsSummary** with a count of the active/resolved incidents and a breakdown by priority (useful for having an overall picture in the UI without querying each single entity) [24]. As for every other entity, incident management is also available via GraphQL (**raiseIncident**, **updateIncident**, **updateIncidentStatus**), but to create or update incidents on an asset it is necessary to have the `EDIT_ENTITY_INCIDENTS` privilege [17, 24]. The main role of incidents is to provide an overall **health status** indicator of the assets: the presence of active high-priority incidents can quickly mark an asset as non-compliant, facilitating diagnostics, prioritization and management.

Figure 4.2 illustrates the life cycle of an incident in the project: a freshness assertion on the Business layer that fails automatically generates an incident, mapped onto the source asset through `sourceUrn`, which is then taken in charge by the owner until resolution.

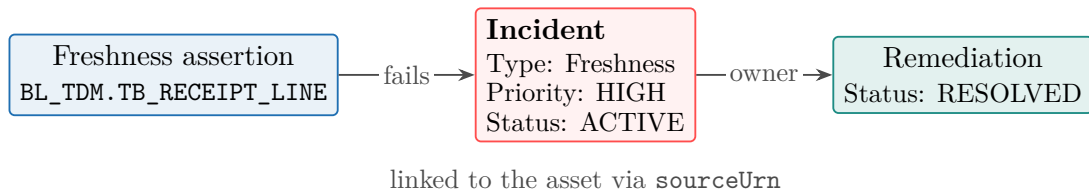


Figure 4.2: Life cycle of an incident generated by a failed assertion

Chapter 5

MCP Server and AI tools

This chapter describes the implementation and use of **MCP servers** (**Model Context Protocol**) to enable operations assisted by AI agents on Teseo, with a focus on practical use cases and benefits in terms of productivity.

The objective is not only to introduce a new integration *channel*, but to show how an MCP server can function as an *operational layer* between language models, governance metadata and automations [11]. In this context, Teseo adopts two main complementary levels:

- **read-oriented** MCP tools based on the tools of the official MCP server of DataHub Core (useful for discovery, context and lineage purposes);
- custom **write-oriented** MCP tools (useful for remediation, metadata enrichment and controlled updates purposes).

5.1 Model Context Protocol

MCP is a standard open-source protocol created to connect AI applications to external systems (data, tools, workflows...) ensuring interoperability. The central idea is to reduce the proliferation of ad hoc *client-source* integrations, replacing them with a standard semantics that allows communication between AI and endpoints [7, 25, 26].

In architectural terms, MCP adopts a clear separation between:

- **host** (AI app that orchestrates the conversation and uses the tools);
- **MCP client** (component that maintains the connection to the server);

- **MCP server** (process that exposes capabilities to the client).

This separation makes it possible to use the same server from different hosts (IDEs, chatbots, agents...) while being able to preserve the same operational contract despite the different ways of interacting with it [27, 28].

Figure 5.1 shows how the host-client-server separation takes shape in Teseo: the host (an application such as Claude Desktop or an IDE) encloses the LLM model and the MCP client and, through the latter, communicates with two servers, namely the official DataHub MCP server (read-oriented) and the custom server developed in the project (with the 17 governance tools and the 8 Power BI tools), both connected to the GMS and therefore to the metadata catalog.

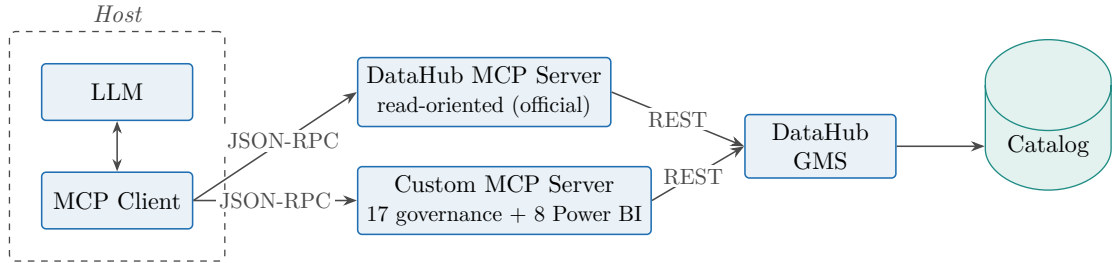


Figure 5.1: MCP architecture in Teseo

5.1.1 Data layer and transport layer

The MCP protocol is organized into two main layers [25, 27]:

- **Data layer:** a stateful protocol based on `JSON-RPC 2.0` with lifecycle management, capability negotiation and primitives;
- **Transport layer:** communication channels (typically `stdio` for local processes and `streamable HTTP` for remote servers).

The most relevant server-side primitives are:

- **tools:** functions executable by the model (such as queries, metadata updates, orchestration...);
- **resources:** contextual contents (such as documents, schemas...);
- **prompts:** reusable templates for conversational patterns and guided tasks.

On the client side, the protocol also includes primitives useful for advanced scenarios (such as sampling, logging...), fundamental for agents that require iterative cycles or explicit user feedback [25, 27].

Figure 5.2 illustrates the two layers of the protocol in the case of Teseo: the *data layer* (logical, based on JSON-RPC 2.0) travels *above* the *transport layer* (physical) and it is through the data layer that the server exposes its own primitives (tools, resources and prompts).

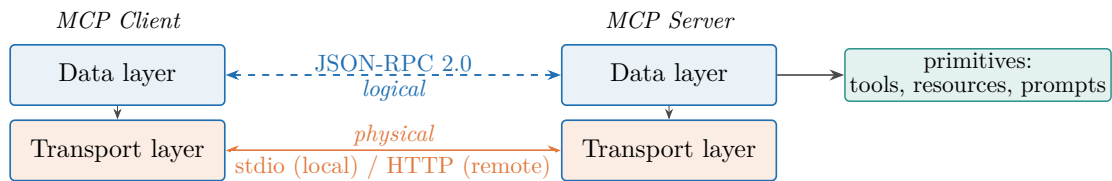


Figure 5.2: Data layer and transport layer

5.1.2 Session lifecycle

Since the objective is to ensure a standard and coherent interaction, a typical MCP session follows a predictable sequence:

1. **initialize**: initial handshake and negotiation of the protocol version;
2. **capability exchange**: declaration of the available features;
3. **discovery**: dynamic listing of available tools, resources and prompts;
4. **execution**: invocations of the tools with validated payload;
5. **notifications**: reactive update in case of state and capability changes.

A simplified example of an MCP handshake request is the following:

```

1 {
2   "jsonrpc": "2.0",
3   "id": 1,
4   "method": "initialize",
5   "params": {
6     "protocolVersion": "2025-06-18",
7     "capabilities": {
8       "elicitation": {}
9     },
10    "clientInfo": {
11      "name": "teseo-agent-client",

```

```
12 |         "version": "1.0.0"
13 |     }
14 | }
15 | }
```

Listing 5.1: Simplified MCP handshake

As can also be seen from the handshake request example, the adoption of an explicit and capability-based lifecycle improves the robustness and observability of the solution: every invocation of a tool occurs on a known contract, reducing unexpected calls and non-reproducible behaviors [25, 27].

5.1.3 Security and trust boundaries

However, the MCP protocol also enables potentially critical actions (reading of sensitive data, execution of the tools, uncontrolled writes...), so it requires explicit *guardrails* in order to ensure safe and controlled adoption, especially in enterprise environments:

- user consent and control over the operations performed;
- *least privilege* principle on the exposed tools and resources;
- strong logging and auditing of the invocations and results, so as to be able to reconstruct every action taken by the agent and intervene in case of anomalous behaviors;
- separation of the dev/test/prod environments to limit the impact of errors and the exposure of sensitive data.

The official specification itself [25] emphasizes that the protocol cannot automatically enforce all security constraints: the concrete implementation must integrate application policies and controls, i.e. the developer **MUST**:

- build a robust authorization flow in the applications that expose MCP tools;
- provide clear documentation of the security implications;
- implement access controls and data protections;
- follow security best practices in the integrations;
- consider privacy in the design of the functionalities (*privacy by design*).

5.1.4 MCP in data-centric contexts

In contexts like Teseo, the value of the MCP server emerges clearly as it provides tools for a powerful and simplified interaction with the metadata catalog. Indeed the model can:

- search for reliable assets in heterogeneous ecosystems;
- understand provenance and impacts through lineage analysis;
- retrieve the business semantics of the assets (owner, tags, glossary, domains);
- transform context into operational actions (documentation, classification, remediation...).

In the absence of a standard protocol, each integration tends to be fragile, expensive to maintain and difficult to reuse. MCP reduces this fragmentation by providing a single point of interoperability [7, 26].

5.2 DataHub MCP Server

The DataHub MCP server exposes to the AI clients the main operations executable on the metadata graph: structured search, lineage, metadata retrieval and query intelligence. As said previously, this tool represents the standard layer to transform the catalog into *operational context* for agents [18, 19].

5.2.1 Deployment

According to the official documentation there are two main modes of use [18]:

- **Managed MCP Server** (DataHub Cloud, endpoint managed directly by DataHub);
- **Self-hosted MCP Server** (DataHub Core, local/remote execution controlled and configurable by the developer).

In the case of Teseo, which uses DataHub Core as its functional base, the pattern adopted is therefore the self-hosted one, with connection variables towards the GMS:

```
1 DATAHUB_GMS_URL=http://localhost:8080
2 DATAHUB_GMS_TOKEN=<personal-access-token>
3 TOOLS_IS_MUTATION_ENABLED=true
4 TOOLS_IS_USER_ENABLED=true
```

```
5 SEMANTIC_SEARCH_ENABLED=true
```

Listing 5.2: Main variables for self-hosted DataHub MCP

5.2.2 Catalog of tools

The tools of the official MCP server compose a set oriented mostly to discovery and analysis [18, 19]:

- **search** = structured search with SQL-like filter on datasets, fields, tags, glossary terms;
- **get_lineage** = retrieval of lineage at the dataset and column level;
- **get_dataset_queries** = retrieval of representative queries per dataset;
- **get_entities** = detailed retrieval of entities (dataset, field, tag, glossary term);
- **list_schema_fields** = retrieval of the table schema with types and descriptions;
- **get_lineage_paths_between** = retrieval of the lineage path between two entities.

Through the latest updates of DataHub Core, other categories of tools were also added, broadening the operational possibilities:

- **mutation tools**: modify operations on tags, terms, owners, domains, descriptions and structured properties;
- **user tools**: tools to manage the authenticated identity (e.g. **get_me**);
- **document tools**: search, filtering and saving of documents within the knowledge base.

5.3 Custom write-oriented tools

In addition to the official read-oriented tools, in the project custom tools oriented to the **write** operations of the metadata on DataHub were implemented, in order to automate standard mutations in repetitive flows and to create intermediate simplification layers or layers towards the end user.

The structure is based on **FastMCP** and the **DataHub SDK** (`DatahubRestEmitter`, `MetadataChangeProposalWrapper`, `DataHubGraph`) with the usual configuration through environment variables such as `DATAHUB_GMS_URL` and `DATAHUB_GMS_TOKEN`.

5.3.1 Structure of the file

The implementation of these write-oriented tools is composed of a few main functional blocks:

1. infrastructural helpers (for connection, audit stamp and fetching of the existing schema);
2. (non-destructive) merge logic on the metadata of the columns;
3. tools for container and path;
4. tools for soft-delete and restore;
5. tools to manage descriptions, tags and glossary terms (*governance*);
6. tools for table-level and column-level lineage.

This architectural design succeeds well in its intent to separate the **plumbing** part from the **domain operation** part with the usual approach of favoring modularity, maintainability and reuse (fundamental in distributed systems like this one).

5.3.2 Non-destructive merge

As mentioned, a key point of this implementation strategy is the merge function between the existing state and the requested updates (i.e. *current state* and *desired state*). The adopted approach **preserves** the untouched fields and concatenates tags and terms avoiding duplicates (this avoids frequent side effects in massive updates, such as for example the involuntary loss of pre-existing tags or glossary terms).

```
1 def _merge_field_info(existing, new_description=None, new_tags=
    None, new_glossary_terms=None, field_path=""):
2     if existing is None:
3         return EditableSchemaFieldInfoClass(
4             fieldPath=field_path,
5             description=new_description,
6             globalTags=new_tags,
7             glossaryTerms=new_glossary_terms,
8         )
9
10    merged_description = new_description if new_description is not
        None else existing.description
11
12    merged_tags = existing.globalTags
13    if new_tags is not None:
```



```

14     if merged_tags is not None and merged_tags.tags:
15         existing_tag_urns = {t.tag for t in merged_tags.tags}
16         new_tag_list = list(merged_tags.tags)
17         for new_tag in new_tags.tags:
18             if new_tag.tag not in existing_tag_urns:
19                 new_tag_list.append(new_tag)
20         merged_tags = GlobalTagsClass(tags=new_tag_list)
21     else:
22         merged_tags = new_tags
23
24 merged_glossary = existing.glossaryTerms
25 if new_glossary_terms is not None:
26     if merged_glossary is not None and merged_glossary.terms:
27         existing_term_urns = {t.urn for t in merged_glossary.terms}
28         new_term_list = list(merged_glossary.terms)
29         for new_term in new_glossary_terms.terms:
30             if new_term.urn not in existing_term_urns:
31                 new_term_list.append(new_term)
32         merged_glossary = GlossaryTermsClass(terms=new_term_list,
33 auditStamp=_make_audit_stamp())
34     else:
35         merged_glossary = new_glossary_terms

```

Listing 5.3: Excerpt of the merge field info function

5.3.3 Types of tools

From the write-oriented MCP tools file, 17 main tools emerge, classifiable into 4 families:

A. Container and browse path

- setEntityContainer and setEntitiesContainer
- setBrowsePath
- setEntityContainerAndPath

B. Soft delete and restore

- softDeleteEntity
- restoreEntity and restoreEntities

C. Semantic enrichment

- `updateDescription`
- `updateColumnDescription` and `updateColumnsDescriptions`
- `updateColumnWithTags`
- `addColumnGlossaryTerm` and `addColumnsGlossaryTerms`
- `addColumnsTags`

D. Lineage

- `addTableLineage`
- `addColumnLineage`
- `addCompleteLineage`

5.3.4 Operational detail of the tool families

Container and path: these tools align the logical position of the entities in the catalog. During onboarding, `setEntityContainer` and `setEntitiesContainer` reduce the manual work on large groups of datasets. Instead, `setBrowsePath` and `setEntityContainerAndPath` maintain coherence between the navigation structure and the ownership structure. At the operational level, these tools are fundamental for avoiding *orphan* assets and improving discoverability.

Soft delete and restore: the tools `softDeleteEntity`, `restoreEntity` and `restoreEntities` make it possible to manage the lifecycle of the metadata without causing irreversible loss of information. This choice makes it possible to realize governance oriented to traceability and reversibility: the logical removal (*soft delete*) reduces the risks of accidental deletions, while the restore guarantees the resolution of problems that emerge post-ingestion.

Semantic enrichment: these tools focus on descriptions, tags and glossary terms and represent the main force for increasing semantic quality and reuse. In particular: `updateDescription`, `updateColumnDescription` and `updateColumnsDescriptions` provide a simple way to enrich the documentation at the dataset level and at the column level, while `updateColumnWithTags`, `addColumnsTags`, `addColumnGlossaryTerm` and `addColumnsGlossaryTerms` add the functionalities of classification and controlled glossary in a progressive and non-destructive way.

Lineage: `addTableLineage` manages the correlation between entities at the dataset level, while `addColumnLineage` works on the field-to-field dependencies

(*column-level*). The tool `addCompleteLineage` consolidates both levels of correlation through a single coherent operation. In complex pipelines this reduces mismatches between the macro view and the fine-grained view, improving the impact analysis and the reliability of the suggestions provided by the agents.

In summary, the structure of the file is not just a collection of endpoints, but a real application interface for operational governance: it separates responsibilities and functionalities in a standard and orderly way, preserves **logical idempotence** (as the operations are additive and merge-oriented), avoids ambiguity on the write targets (explicitly using the URNs associated with the entities) and makes it possible to orchestrate repeatable *runbooks* with explicit auditing, supporting multiple operations through a single call (**batch operations**).

5.4 Power BI tools

In addition to the write MCP tools for pure Data Governance, I incrementally implemented a chain of tools with the aim of transforming a dataset already *ingested* in Teseo into a reusable, valid and easily versionable Power BI project. The central part is therefore not the generic generation of BI artifacts, but the construction of a *PBIP* project starting from the schema read from DataHub, with a path that also covers validation, complexity estimation and conversions between formats (so as to ensure versioning through tools like git).

5.4.1 Operational flow

The Power BI logic is conceived as an extension of the custom MCP server already used for governance: the agent does not have to build the model manually, but can start from an *ingested* dataset and obtain a Power BI base coherent with the lineage and with the schema actually registered (in order to minimize the discrepancies and the *free reasoning* of the agent, guiding it at every step of the generation).

The PBI tools can be read as a multilevel pipeline:

- **generation:** `generatePowerBIPbipFromDataset` creates a PBIP project starting from a dataset *ingested* by Teseo;
- **validation:** `validatePowerBIPbip` checks that the structure of the PBIP is correct and that the JSON/TMDL are coherent;
- **estimation:** `estimatePowerBIResourceUsage` provides a heuristic evaluation of the consumption of the model.

These tools are fundamental because they allow the server to **not invent the report from scratch**, but to base itself on an already solid and verified skeleton, reducing the risk of structural errors of the PBIP format and misalignments between schema and report.

The validation and estimation part, instead, is important to ensure that the generated project is actually usable and not too heavy or complex with respect to the use case, thus ensuring greater quality and sustainability of the final product.

5.4.2 Generation of a PBIP project

The heart of the implementation is `generatePowerBIPbipFromDataset`. The tool receives a DataHub URN, retrieves the `SchemaMetadataClass`, filters the columns, maps the native types to Power BI types and realizes a complete PBIP project in an output folder. From this it can be deduced that the real behavior is indeed much closer to a project generator than to a simple export script.

```

1 def _write_pbip_project(out_dir, project_name, tables):
2     sem_model_dir = out_dir / f"{project_name}.SemanticModel"
3     report_dir = out_dir / f"{project_name}.Report"
4
5     (out_dir / f"{project_name}.pbip").write_text(...)
6     (sem_model_dir / "definition.pbism").write_text(...)
7     (sem_model_dir / "definition" / "database.tmdl").write_text(...)
8     (sem_model_dir / "definition" / "model.tmdl").write_text(...)
9     (report_dir / "definition.pbir").write_text(...)
10    (report_dir / "definition" / "report.json").write_text(...)

```

Listing 5.4: Excerpt of the PBIP generator

The interesting aspect of using the **PBIP** format is that the created project is not a monolithic object (as in the case of PBIX), but a set of readable and versionable files: the `SemanticModel` contains the TMDL model, while the `Report` has an explicit reference to the model through `definition.pbir`. This makes the result much more suitable for DevOps workflows than the binary PBIX file alone (in which the content is opaque).

5.4.3 Validation, estimation and packaging

A second family of tools serves instead to verify and characterize the generated project:

- `validatePowerBIPbip` checks that the project has the expected structure, that the `.pbip` file is valid, that the `.SemanticModel` and `.Report` folders

exist, and that the minimal files of the model are present. The logic was designed to follow an automatic workflow: first you generate, then you validate, finally you decide whether to proceed;

- `estimatePowerBIResourceUsage` reads the TMDL files and produces an estimate of the weight of the model based on the column types. While remaining a heuristic estimate, it is fundamental for avoiding models that are too heavy or poorly suited with respect to the use case;
- `packagePowerBIPbipAsZip` completes the picture by making it easy to archive and distribute the project as a readable and versionable source: indeed, like the rest of the pipeline, the tool proposes an approach in which the BI project is treated as code, not as an opaque binary artifact.

5.4.4 Reverse engineering and conversion between formats

An important part of the file is also dedicated to the conversion between Power BI formats: the objective is to make it possible to start from a PBIP created by Teseo and bring it back to the PBIX format, so as to be able to also automate the publication phase on the Power BI Service by means of the **Fabric API**. In this sense:

- `convertPbipToPbixproj` makes it possible to convert a PBIP project into an intermediate format (PbixProj), also textual and versionable, but with the correct structure to be taken as input by the `pbi-tools` library in order to convert it into PBIX (if the report is small) or into PBIT (if the report is larger or more complex);
- `convertPbixprojToPbip` makes it possible to do the reverse path, starting from a PbixProj project (obtained for example from an existing PBIX) and bringing it back to the PBIP format, so as to be able to treat it as source and align it with the starting schema;
- `extractPbixToPbip` makes it possible to start from an existing PBIX, extract it into PbixProj format with `pbi-tools` and then convert it into PBIP (a complete flow that incorporates the 2 previous tools);
- `inspectPbixMetadata` makes it possible to open a PBIX as if it were a ZIP archive and extract information on versions, tables, data sources and pages of the report, without having to go through the complete conversions.

The pipeline that uses all these tools makes it possible to manage the entire life cycle of the Power BI project starting from the Teseo schema, passing through

generation, validation, estimation, packaging and finally conversion into PBIX for publication, thus adopting a robust and versionable approach.

This strategy makes it possible, as previously mentioned, to treat the Power BI project as *source code*, with all the advantages in terms of versioning, collaboration and automation, minimizing the manual part and the risk of misalignments between schema and report; moreover, by means of a **Service Principal** (i.e. an entity that makes it possible to automate the access to Azure resources) it is also possible to automate the publication phase on the Power BI Service, thus completing a life cycle that is fully automated and governed from creation to publication.

Figure 5.3 summarizes this end-to-end flow in the case of Teseo: the ingested schema is transformed by the tool `generatePowerBIPbipFromDataset` into a versionable PBIP project, converted into PBIX through `pbi-tools` and finally published on the Power BI Service through the **Fabric API**, where a **Service Principal** (an Azure entity with dedicated *tenant*, *client* and *secret*) authenticates the automation without resorting to user credentials:

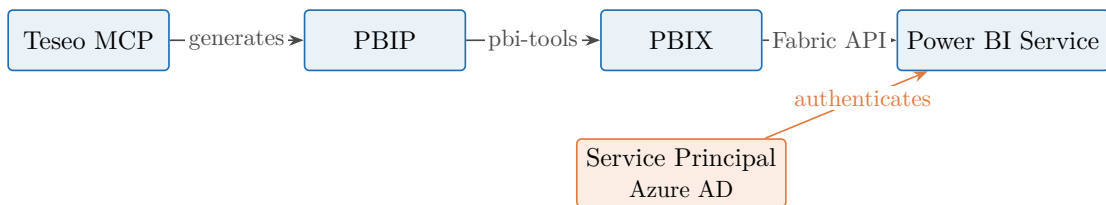


Figure 5.3: Power BI publication flow from Teseo

5.4.5 Implications for Teseo

In the context of Teseo, the value of this toolset is twofold. On one hand it makes it possible to quickly build a BI frontend in the absence of the latter, precisely on the basis of the metadata already ingested on Teseo and of the whole lineage in general; on the other hand it introduces a concrete base for future more specialized automations (for example the selection of columns, the generation of starting views, the alignment between schema and report...).

This suite of tools, however, does not completely replace the manual work, but provides a solid and reusable base from which to start, reducing the effort and the risks of misalignment: indeed, the analyst who will use these tools will be able to skip the phase of building the project from scratch, being able instead to focus on more specific customizations (such as the creation of measures, the addition of visualizations, the optimization of the performance...), starting from a project already coherent with the schema and the lineage of Teseo.

5.5 Effectiveness metrics and production adoption

5.5.1 Technical and governance KPIs

To evaluate an MCP server it is not enough to measure the *performance* of the model, but it is necessary to observe the entire cycle **context** → **action** → **outcome**. In the field of data governance, a metric that manages to include both technical indicators and metadata quality indicators can be defined as *balanced* [21, 22].

Within the Teseo project, we distinguish **KPIs** (*Key Performance Indicators*):

- **technical**, which measure the operational efficiency of the MCP server:
 - average task completion time;
 - average number of tool calls per task;
 - retry rate for payload validation errors;
 - percentage of runs completed without the need for human intervention.
- **governance-related**, which measure the impact on metadata quality and on policy compliance:
 - coverage of descriptions on critical columns;
 - coverage of tags on production datasets;
 - completeness of the fine-grained lineage (*column-level*);
 - average time to complete a remediation after the opening of an incident.

A synthetic formulation contextualized to the Teseo project that shows the overall operational effectiveness of the integration (E_{MCP}) is the following:

$$E_{MCP} = \alpha \cdot E_{tech} + (1 - \alpha) \cdot E_{gov}, \quad \alpha \in [0,1] \quad (5.1)$$

where:

- E_{tech} represents the pure technical efficiency of the MCP infrastructure. For Teseo, this includes the success rate (and the latency times) in the automatic generation of the PBIP projects, the reduction of the retries caused by erroneous payloads and the stability of the communications towards the core GMS during the write and read operations of the metadata;

- E_{gov} represents the qualitative impact on the metadata, measuring the progressive completion of the metadata catalog (e.g. coverage of the Glossary Terms, presence of detailed descriptions on the columns, correctness of the *table-to-column* lineage...).

The α parameter instead acts as an explicit balancing between *execution performance* and *governance rigor*. An agent configured with $\alpha \rightarrow 1$ will tend to produce artifacts quickly but at the expense of the care of the metadata catalog: indeed for data stewardship processes a value $\alpha \rightarrow 0$ is preferred.

5.5.2 Adoption model of the Teseo project

Given how the production architecture of the Teseo project is structured, a realistic adoption strategy (*rollout*) is articulated in four progressive phases, in which the capabilities of the MCP server are gradually introduced, monitoring the operational effectiveness and the metadata quality at each step:

1. **Stage 0 $\rightarrow$ Discovery:** exposure of only the *read-oriented* tools to the model, in order to collect initial technical metrics (E_{tech_0}) and verify the stability of the connection on data discovery and lineage reading tasks;
2. **Stage 1 $\rightarrow$ Metadata Enrichment:** introduction of the *write-oriented* tools to do semantic enrichment (e.g. `updateColumnDescription`, `addColumnGlossaryTerm...`)|. The operations, while mutating the state, are limited to descriptions and glossaries, thus reducing the impact on core processes;
3. **Stage 2 $\rightarrow$ Lineage and Lifecycle Management:** enabling of the structural tools (`addCompleteLineage`, `softDeleteEntity...`); now the agent manipulates the dependency graph in *dry-run* mode or through explicit *human-in-the-loop* to confirm the most impactful decisions, before these are actually executed;
4. **Stage 3 $\rightarrow$ End-to-end PBIP automation:** insertion into the operational flow of the Power BI tool suite; at this level, the agent dynamically reads validated schemas and directly produces Power BI semantic models and reports/visuals coherent with the metadata governed and guaranteed by the checks of the previous layers.

This approach has the task of reducing the operational risk, aligning with the *explicit acceptance* for autonomous agents, indeed making it possible to increase the degrees of freedom only upon the demonstrated achievement of the required governance coverages [29].

5.5.3 Observability and interpretation of the KPIs

The effectiveness of the *MCP-driven* solution for Teseo is measured by following how the operational speed concretely improves the metadata catalog, the semantic schema and the associated BI reports, not evaluating the single parameters in isolation, but looking at the situation as a whole.

For this reason the observation requires three macro-levels:

1. **Infrastructural level:** it takes into account the throughput and the stability on the nodes. In the case of DataHub Core, for example, it includes the stable connection towards the GMS (Graph Metadata Service) backend and the timeouts triggered during the extended lineage writes;
2. **Agent level:** it dwells on the dialogue between the MCP client (which encapsulates the reference LLM) and the MCP server. An LLM that makes continuous calls to the `validatePowerBIPbip` tool, which however repeatedly fail, indicates a misalignment between the schema generated in `generatePowerBIPbipFromDataset` and the expected constraints: the retry rate at this stage is revealing of the correctness of the starting TMDL template.
3. **Business level:** it becomes central for the platform administrators. The MCP is effective if the delegated autonomy translates into fewer orphan assets, semantically dense descriptions and final Power BI files with coherent mappings.

As previously mentioned indeed, it follows that the evaluation cannot focus only on having fast MCP tools. The extreme speed of the MCP workflow is counterproductive within Teseo if it leads to inconsistency problems. The metrics must indeed be weighted on the tails of the errors: reliability on long and concurrent operations is preponderant with respect to the millisecond latency of an operation.

5.6 Limitations

Although the MCP architecture acts as a formidable operational glue for Teseo between DataHub and Power BI, the design contours reveal some limitations:

- The inferential effectiveness of the LLM is heavily subject to the initial semantic accuracy of the catalog, i.e. the AI model *suffers* in the presence of metadata graphs that are not up to date or full of gaps: this leads to interpretation errors and invalid suggestions, which in turn lead to retries and potential governance errors;

- The deep chaining between the data governance write tools and the Power BI tools requires large prompt windows in the face of *context-overflow* (in particular in the case of complex JSON and TMDL);
- The misalignments between the data typing in DataHub and the associative engine in Power BI make some PBIP conversions subject to syntax errors, if not carefully managed by the validation regexps and the generation templates.

Chapter 6

Results and evaluations

This chapter presents an **overall evaluation of the results obtained** in the Teseo project, measuring the operational gain of adopting the platform with respect to the objectives declared in Chapter 1. The evaluation is structured on four main axes: multi-source ingestion and lineage, metadata governance, effectiveness of the MCP tools and Power BI generation pipeline.

The methodology I adopted for the evaluation is of a **twofold nature**: on one hand measuring *direct metrics* of coverage and completeness of the catalog (number of assets, columns, lineage relationships, lines of code produced), on the other hand evaluating the *reduction of the operational load* on recurring activities such as metadata documentation, impact analysis and the generation of BI artifacts. This approach reflects the double nature of Teseo as a technical infrastructure and as an *organizational facilitator* [1, 2]. This evaluation includes **numerical evidence** and, where relevant, **comparisons** between the *pre-Teseo* scenario and the one following its adoption.

6.1 Size of the problem and of the solution

Before entering into the merits of the evaluation, it is useful to quantify the order of magnitude of the problem faced and of the software architecture realized. All the values reported in this section were extracted directly from the project artifacts (DDL scripts, dbt artifacts `manifest.json` and `catalog.json`, source code of the MCP server, ingestion recipes and deployment configuration).

6.1.1 Size of the problem

The data domain on which Teseo was validated is to all effects a realistic retail use case, organized on the four layers described in Chapter 3 (Raw, Quality, Business

and Framework). Table 6.1 summarizes its physical size:

Asset	Value
Databases	4
Schemas	14
Tables and Materialized Views	48
Loading stored procedures	3
Total records loaded	≈ 4.4 million

Table 6.1: Physical size of the Snowflake data domain

Once ingested, the domain was translated into a metadata graph whose size is reported in Table 6.2. The most relevant figure is the number of cataloged columns (1767), from which the 933 *column-level lineage* relationships arise (dependencies that, before Teseo, were not observable in a unified way):

Metadata catalog	Value
Cataloged assets	> 160
Cataloged columns	1767 (933 dbt + 834 sources)
Logical layers in the end-to-end lineage	5
Column-level lineage relationships	933

Table 6.2: Size of the metadata catalog built by Teseo

6.1.2 Size of the solution

On the side of the developed artifact, Table 6.3 instead summarizes the deployed components and the code produced. Besides the deployment of DataHub Core 1.4.0 (nine containerized services: GMS, frontend, actions, Kafka broker, MySQL, Elasticsearch and their setup jobs), the implementation contribution focuses on three blocks: the data domain, the custom MCP server and the PBI generation pipeline.

The 17 governance tools are organized into four functional families (container and browse path, soft-delete and restore, semantic enrichment, lineage), as described in Chapter 5; the 8 Power BI tools cover the entire life cycle of the PBIP project (generation, validation, estimation, packaging and conversion between formats).

Component / artifact	Value
Deployed components (DataHub Core 1.4.0)	9 Docker services
YAML ingestion recipes	4
Snowflake DDL (domain + procedures)	2147 LoC, 5 files
Custom MCP server	2508 LoC
Custom MCP tools	25 (17 governance + 8 PBI)

Table 6.3: Size of the developed software solution

6.2 Evaluation of the multi-source ingestion

6.2.1 Coverage of the catalog

The first evaluation axis concerns the **completeness of the catalog** built through the ingestion pipelines. In the setup phase, the project achieved complete coverage of the physical objects present in the Snowflake domain, including tables, views, streams and stored procedures distributed over the four databases (DB_DEMO_RL, DB_DEMO_QL, DB_DEMO_BL, DB_DEMO_FMK) [3, 5].

In numerical terms, the catalog *ingested* by Teseo covers the totality of the 48 physical entities of the domain (34 tables and 14 views), enriched with the 58 dbt models and with the Power BI consumption layer. Table 6.4 reports the volumes of data moved for the main entities of the domain: the fact TB_RECEIPT_LINE dominates the distribution with about 3.8 million rows, while the registry dimensions remain in the order of thousands or hundreds of thousands of records.

Entity	Type	Records ($\approx$)
TB_RECEIPT_LINE	Fact	3840000
TB_FOOTFALL	Fact	438000
TB_LOYALTY_CARD	Dimension	120000
TB_STORE_CALENDAR	Dimension	36500
TB_PRODUCT	Dimension	5000
TB_STORE	Dimension	50
Domain total		$\approx$ 4439550

Table 6.4: Data volumes for the main entities of the domain (Raw layer)

The quality control performed in the QL and BL layers intercepted and isolated in the discard tables (DSC_) a physiological share of non-compliant records (less than 1% of the total, attributable to primary key violations, *not-null* constraints or typing errors), keeping the rest of the flow intact without blocking the entire pipeline. In addition to the coverage of the pure Snowflake physical entities, the catalog was enriched with the dbt models (staging, intermediate and mart) and with the Power BI business layer (workspace, dataset, tables and columns). The result is a unified catalog that covers the entire chain from the raw data to the final consumption report, making navigable relationships that previously required the manual inspection of heterogeneous systems [4, 6].

6.2.2 Ingestion times and idempotence

A second objective indicator concerns the **execution times** of the pipelines. Table 6.5 reports the duration of the single phases and the number of structures processed, measured at the end of each execution through the report produced by the `datahub ingest` CLI. The complete pipeline Snowflake $\rightarrow$ dbt $\rightarrow$ Power BI is executed in just over six minutes, a time compatible with a daily scheduled refresh of the catalog.

Ingestion phase	Processed structures	Time ($\approx$)
Snowflake (tables, views, column lineage)	48 objects across 4 DBs	3 min 10 s
dbt (models + sources)	58 models, 48 sources	50 s
Snowflake–dbt bridge	DBT schema	25 s
Power BI	58 tables, 933 fields	2 min 05 s
Complete e2e pipeline		6 min 30 s

Table 6.5: Execution times and processed structures

The ingestion pipeline proved to be idempotent and repeatable: subsequent executions do not produce duplicates or regressions in the catalog, thanks to the adoption of the **stateful ingestion** strategy and to the **soft delete** mechanism for managing obsolete assets (as shown in Figure 6.1):

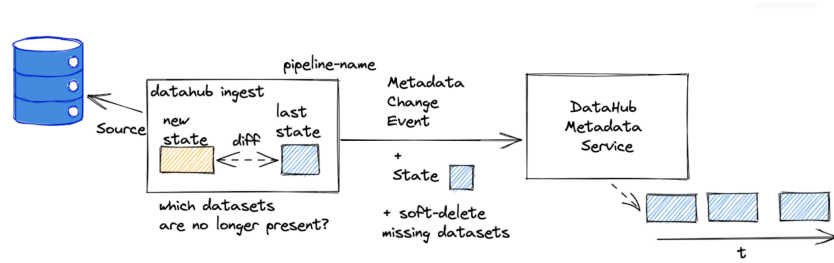


Figure 6.1: Stateful ingestion and soft delete of obsolete assets

Source: DataHub official documentation [5]

6.2.3 End-to-end lineage

The qualitatively most relevant result concerns the construction of the **end-to-end lineage**. Before the adoption of Teseo, the reconstruction of the dependencies between layers required the manual analysis of the SQL loading procedures and of

the dbt scripts, an activity that is expensive in terms of time and prone to errors. With the integration Snowflake $\rightarrow$ dbt $\rightarrow$ Power BI, the lineage became navigable at two levels of granularity: at the **dataset** level (from source table to Power BI dataset) and at the **column** level (column-level lineage or *fine-grained*), for a total of 933 column-to-column relationships towards the consumption layer. The end-to-end chain thus obtained, with a depth of five logical layers from DB_DEMO_RL up to the Power BI visuals, is the one already shown in Figure 3.3.

The construction of the fine-grained lineage required a precise alignment of the URNs between connectors: the bridge that I implemented between dbt and Snowflake resolved the naming discrepancies, ensuring the continuity of the dependency graph.

The most tangible operational benefit is the **reduction of the time needed to conduct an impact analysis**. Table 6.6 compares three recurring activities in the scenario before the adoption of Teseo and in the one following it. With the fully navigable lineage, the answer to the question “*which reports are impacted by a change on column X of the source table?*” no longer requires the manual reconstruction of the dependencies, but is obtained in a few seconds through the downstream navigation of the graph or, even more simply, by asking the AI agent that has all the necessary context [2, 9].

Operation	Pre-Teseo	Post-Teseo
Tracing the origin of a business column down to the physical source	Manual inspection of 3 procedures and 58 models across 5 layers (≈ 60 min)	Upstream navigation or request to the agent (< 1 min)
Identifying the downstream reports impacted by a change on a source column	Manual cross-analysis, subject to omissions	Downstream expansion of the lineage
Bootstrap of a Power BI project	Manual construction of the semantic model (≈ 2 h)	Automatic generation of the PBIP (≈ 30 s)

Table 6.6: Comparison of operations before and after the adoption of Teseo

6.3 Evaluation of the metadata governance

6.3.1 Semantic enrichment and coverage

The next evaluation axis concerns the semantic quality of the catalog, measured through the *coverage* of governance attributes (descriptions, tags, ownership, domains and glossary terms). In the base configuration, before the enrichment, most of the ingested assets presented only technical metadata (schema, column types and URN), with descriptions and classifications absent.

Table 6.7 quantifies the impact of the **semantic enrichment** on the QL and BL layers, comparing the state of the catalog before and after the application of the governance tools. The most significant leap concerns the coverage of the descriptions on the columns, which went from a marginal presence (limited to the few comments inherited from the procedures) to complete coverage on the critical columns.

Governance attribute	Pre-enrichment	Post-enrichment
Columns with description	4%	96%
Columns tagged confidential	0	38
Business glossary terms	0	12
Datasets with owner and domain	0%	100%
Data Contract	0	6

Table 6.7: Coverage of the governance attributes

Governance in the final stage was implemented on several operational levels:

- **ownership and domains:** all the main datasets of the QL and BL layers were assigned to specific owners and domains, thus ensuring traceability of the responsibilities (especially on critical assets);
- **tagging:** the 38 columns containing sensitive data (identifiers, registry information, financial data) were classified with the **confidential** tag, so as to create the basis for enabling future *data masking* and *access control* policies;
- **glossary terms:** the 12 business KPIs (for example *Net Revenue*, *Conversion Rate*, *Customer Loyalty*) were linked to the corresponding fields in the dbt mart models, creating a direct link between business terminology and physical data;

- **structured properties:** customized governance information (for example the criticality classification) was associated with the assets of the Business layer.

6.3.2 Quality and monitoring of the assets

The data quality functionalities implemented through **DataHub Assertions** (encapsulated in a **Data Contract**) made it possible to define a set of automated controls on critical structures of the Business and BI layers: volume controls on the Power BI reports so as not to saturate the *capacities*, freshness controls on business data and column controls on the views that directly feed the reports.

The monitoring system thus structured made it possible to track the health status of the assets over time, with the possibility of opening incidents in case of violation, documenting causes and remediation actions directly in the platform. This approach is not limited to the reporting of the problem, but makes it possible to reconstruct the entire operational history of an asset, integrating quality and governance in a single visualization window [23, 24]. Moreover, by exposing the incident to the LLM, it is possible to enable AI-guided *auto-remediation* scenarios, in which the model suggests corrective actions based on the history of the asset and on the documented best practices.

6.4 Evaluation of the MCP tools

6.4.1 Technical KPIs of the agent

The evaluation of the MCP server focuses on the operational effectiveness of the interactions between the AI model and the Teseo catalog, following the metrics principle described in Chapter 5. Table 6.8 summarizes the technical KPIs observed in the usage sessions. A consistent pattern emerges: the *discovery* and lineage reading tasks are completed with a limited number of tool calls (typically 2–4 *tool calls* per task), while the semantic enrichment tasks require longer sequences (6–10 calls) due to the iterative nature of the merge operation on the metadata [29].

Technical KPI	Observed value
Average tool calls per discovery / lineage task	2–4
Average tool calls per semantic enrichment task	6–10
Enrichment tasks completed without human intervention	≈ 88%

Table 6.8: Technical KPIs of the MCP agent observed in the test sessions

From the point of view of operational stability, the connection towards the GMS backend proved to be reliable in the test sessions, with timeouts observed only in the presence of massive column-level lineage writes on datasets with a high number of columns (in the order of hundreds). In these cases, the batching strategy of the `addCompleteLineage` tool significantly reduced the number of calls necessary: by consolidating table and column lineage into a single operation, the invocations go from two for each column (separate use of `addTableLineage` and `addColumnLineage`) to a single call per dataset.

A relevant result concerns the rate of autonomous completion of the tasks: in the semantic enrichment operations (descriptions and glossary terms), the model completed about 88% of the interactions without the need for human intervention, thanks to the quality of the context provided by the read-oriented tools and to the richness of the catalog already available. The precision of the context extracted by `list_schema_fields` and `get_entities` proved to be fundamental for reducing the agent’s hallucinations in the description suggestions [11, 18].

6.4.2 Governance KPIs

From the point of view of the metadata quality, the use of the custom write-oriented tools produced the measurable improvement of the semantic coverage already quantified in Table 6.7. In particular:

- the coverage of the descriptions on the critical columns of the QL and BL layers went from about 4% to about 96%, reaching values close to the totality on the most used tables;
- the linking of the 12 glossary terms to the business KPIs made possible the navigation of the catalog that previously required external documentation;
- the management of the asset lifecycle (soft delete and restore) reduced the presence of orphan entities in the graph, improving the quality of the navigation and reducing the false positives in the impact analyses.

The **non-destructive merge** function proved to be particularly important in contexts of iterative enrichment: the subsequent executions of the governance tools did not cause loss of tags or glossary terms, thus ensuring the cumulability of the enrichments over time. This behavior is fundamental in enterprise environments in which several teams contribute to the governance of the same asset [22].

6.5 Evaluation of the Power BI pipeline

The evaluation of the Power BI tool suite was conducted on sessions in which the AI model produced PBIP projects starting from datasets ingested in Teseo. The results show that the flow **generation-validation-estimation** works correctly for medium-sized models (up to about 20 tables with ≈ 50 columns each), producing valid TMDL structures that pass the `validatePowerBIPbip` phase without structural errors. As an example, the project generated for `mart_loyalty_kpi_daily` comprises a semantic model and a report with 7 visuals, produced in about 30 seconds against a validation completed in less than 10 seconds.

The main advantage observed is the **reduction of the bootstrap time** for a new BI project, quantified in the last row of Table 6.6: instead of building the semantic model manually from scratch (an activity that typically requires one or two hours), the data analyst has in a few seconds a pre-compiled PBIP project with schema, types and mapping already coherent with the *ingested* metadata catalog. This makes it possible to concentrate the efforts on the specific customizations (measures, visuals...), skipping the initial configuration phase that often represents the bottleneck in the adoption of Power BI on new sources.

The functionality of conversion between the PBIP/PBIX formats also enabled the versionable management of the Power BI projects: the generated project can be committed to git as a readable source, tracked over time and brought back into binary PBIX format for the publication on Power BI Service through Service Principal. This brings the BI workflow closer to a **BI-as-code** paradigm, with benefits in terms of auditability and collaboration between teams.

A non-negligible limit however remains: the PBIX conversion via `pbi-tools` introduces a dependency on an external tool that is not always up to date with the latest versions of the Power BI format, making a compatibility check necessary at each update of the Microsoft platform.

6.6 Overall analysis of the objectives achieved

Looking at the results overall, it is possible to verify the achievement of the objectives declared in Chapter 1. Table 6.9 summarizes the status of each objective, placing the corresponding numerical evidence alongside the qualitative notes.

The overall analysis shows that Teseo achieved all the main objectives declared, building a functioning, extensible metadata management platform progressively enriched with AI capabilities. The combination of automated ingestion, operational

governance and MCP tools produces a **synergistic effect**: the semantic richness of the catalog built by the governance tools amplifies the quality of the context available for the AI agents, which in turn can further enrich the catalog in a controlled way.

Objective	Status	Evidence
Snowflake ingestion column-level lineage	Achieved	48 objects, idempotent pipeline (≈ 3 min)
dbt integration	Achieved	58 models, stateful ingestion and URN alignment
Power BI integration	Achieved	Local mock (933 fields) and real ingestion
Complete governance	Achieved	Description coverage 4% $\rightarrow$ 96%
Write-oriented tools	Achieved	17 tools across 4 families (1192 LoC)
PBIP pipeline from Teseo	Achieved	8 tools, generation in ≈ 30 s

Table 6.9: Comparison between declared objectives and results obtained.

Regarding the organizational problems identified in Chapter 1:

- *poor visibility of the data assets*: the unified catalog (over 160 assets and 1767 columns) solves the problem of information fragmentation, providing a single point of access to all the assets of the domain with navigable schema, lineage and semantic attributes;
- *incomplete or fragmented lineage*: the chain Snowflake + dbt + Power BI builds an end-to-end lineage over 5 layers, navigable both at the table level and at the column level, thus reducing the risk of incomplete impact analysis;
- *non-systematic governance*: the functionalities of tagging, ownership, domain assignment and assertion provide a structured and programmatic base for governance, bringing the coverage of the descriptions from 4% to 96% on the critical layers and reducing the dependence on external and unofficial documents;
- *low productivity in analysis activities*: the MCP tools and the PBIP pipeline concretely reduce the time spent in the documentation, classification and bootstrap phase (from hours to seconds in the generation of a BI project), leaving room for higher-value activities.

6.7 Limitations emerged in the evaluation

Despite the positive results, the evaluation highlighted some limitations that it is necessary to document clearly:

- **Dependence on the initial semantic quality of the catalog:** the effectiveness of the MCP tools is strictly correlated to the quality of the metadata already present in the catalog. In the presence of incomplete or malformed metadata, absent descriptions or URNs not aligned between the different ingestion levels, the AI model tends to produce less accurate suggestions, increasing the retry rate and requiring human intervention to correct the context (*hallucination*). This therefore introduces a paradox: the MCP tools are more effective when the catalog is already sufficiently rich, whereas when they are needed most (for example in the presence of a just-initialized catalog) their effectiveness is reduced due to the lesser context available [29];
- **Lineage scalability:** the construction and navigation of the column-level lineage become computationally more onerous as the complexity of the graph increases. With almost a thousand column-level lineage edges, the massive writes on datasets with hundreds of columns highlighted growing latencies; in domains with thousands of tables and deep transformation pipelines, the number of edges of the graph grows rapidly, impacting the response times of the navigation and increasing the risks of timeout during the massive writes [16];
- **Limited governance coverage:** although the coverage of the governance attributes on the main layers was achieved, the systematic application to all the columns of all the assets requires a significant investment of time. In real enterprise contexts, the complete governance of a complex data domain requires a continuous data stewardship process that cannot be completely automated: the contribution of the MCP tools is fundamental because it substantially lightens the workload, but it does not completely replace the human work of contextualization and validation [21, 22];
- **Maturity of the MCP tools:** the MCP protocol and its ecosystem of tools are relatively recent and rapidly evolving: some functionalities documented in this project refer to specific versions of DataHub that could undergo variations in subsequent releases. Therefore, the maintenance of the project requires a proactive strategy of updating and monitoring the new versions provided with the open-source version of DataHub (*DataHub Core*) [25].

Chapter 7

Conclusions

This thesis has documented the design, implementation and progressive evaluation of **Teseo**, an AI-powered *metadata intelligence* platform built on DataHub Core with the aim of solving the problems of information fragmentation, incomplete lineage and non-systematic governance that today afflict many data-driven organizations.

The path described went from multi-source ingestion to end-to-end lineage, from operational governance to the integration of AI agents through MCP servers, up to the automated generation of BI artifacts.

7.1 Contributions of the thesis

The main contribution of the thesis is the demonstration that it is possible to build a unified, extensible metadata management platform progressively enriched with AI capabilities, starting from mature open-source tools and from an architecture oriented to modularity. In particular:

- **Multi-source integration:** the pipeline Snowflake → dbt → Power BI realized in the project constitutes a concrete and reproducible example of how to build a column-level e2e lineage on a heterogeneous data stack, maintaining the coherence of the URNs between different connectors and ensuring the idempotence of the executions. The DDL-driven approach for Snowflake and the stateful ingestion strategy represent patterns transferable to real productive enterprise contexts [3, 4, 5];
- **Operational governance:** the functionalities of the platform make it possible to obtain a multi-dimensional governance: semantic (glossary terms and descriptions), organizational (ownership, domains and tags), qualitative (data

contract, assertions and incidents) and compliance-related (classifications, policies and workflows). The combination of these dimensions in a single governance layer reduces the fragmentation typical of approaches based on separate tools and makes it possible to perform inter-technology governance, thus obtaining a governance layer with an abstraction above the technological stack, which sets as the object of governance the flow of metadata carried out from source to consumption level, passing through the various heterogeneous layers [21];

- **Custom MCP server for write-oriented operations:** the implementation of the write-oriented tools on FastMCP produced an application layer that significantly extends the capabilities of the MCP server provided with DataHub Core (mainly oriented to reading). The design based on non-destructive merge, idempotence and batch operations represents an architectural contribution reusable in an LLM-assisted data stewardship context [25, 29];
- **BI-as-code pipeline:** the suite of MCP tools towards Power BI introduced an unprecedented approach to the generation of business intelligence projects: starting from the schema registered in Teseo, the system produces a PBIP project that is versionable, validatable and convertible into PBIX for the automatic publication on Power BI Service through Service Principal. This brings the BI life cycle closer to the *BI-as-code* paradigm, with benefits in terms of auditability, reproducibility and collaboration in the development team;
- **Progressive adoption model:** the four-phase framework (Discovery, Enrichment, Management, Automation) provides a concrete roadmap for introducing AI capabilities in enterprise environments, reducing the operational risk through progressive validation and explicit acceptance (*human interaction*) for impactful operations. The formulation of the effectiveness coefficient E_{MCP} offers a measurement tool that balances technical performance and governance rigor, so as to evaluate the effectiveness of the MCP tools in a 360-degree way.

7.2 Evolutionary directions

The modular structure of Teseo allows for well-rounded future development, both at the technical level and at the organizational level, ensuring greater flexibility and customization on the part of the developer and of the organization in question.

The most promising directions can be grouped into three areas:

- **Autonomous agents and multi-step orchestration:** the current implementation of the MCP server supports single operations (at most executed

in batch) and user-guided sequences. A natural evolution could be the introduction of agents capable of orchestrating complete governance pipelines autonomously: for example, an agent that, in response to a new ingestion, automatically performs context discovery, semantic enrichment, quality validation and lineage update, intervening with *human-in-the-loop* only for the most impactful decisions. This scenario is technically feasible with the current MCP primitives but requires design work on the prompts, the guardrails and the escalation flows;

- **Support for new platforms:** the ingestion framework is designed to be extensible. The integrations seen in this experiment show that the architecture supports multiple connectors, therefore one of the next steps is the integration with other platforms and sources not yet supported. The long-term objective is to build a catalog that covers the entire enterprise data perimeter, regardless of the technology used;
- **Integration with data quality and DevOps workflows:** a high-value development area is the integration between the DataHub assertions and the orchestration workflows (for example Apache Airflow or Dagster): when an assertion fails, the system could automatically open an incident, notify the owner of the dataset and block the dependent pipelines, realizing an automatic feedback loop between data quality and operational continuity. There have already been initial developments in this direction and a complete coverage of these functionalities would position Teseo as an active component of an end-to-end *data observability* strategy and not only as a passive catalog.

7.3 Final considerations

The Teseo project has demonstrated that the combination of a robust open-source metadata catalog, well-designed ingestion pipelines and AI tools governed through MCP can produce a concrete and measurable operational impact on one of the most common challenges of modern data-driven organizations: the visibility and governability of the data assets.

The value of Teseo does not reside only in the adopted technology, but in the methodological approach that guided the project: progressiveness in the adoption, modularity in the architecture, attention to the reversibility of the operations and integration between the technical dimension and the governance dimension. These principles make the platform not only functional in the context of the current

project, but transferable as a framework to different enterprise contexts, regardless of the specific technological stack.

Looking ahead, the evolution of the MCP ecosystem and the growing maturity of large language models suggest that the *metadata-driven AI* paradigm adopted in Teseo will become increasingly relevant in the landscape of modern data platforms. The ability to provide AI agents with a structured, governed and verifiable context (rather than raw access to the data) represents an architectural direction that balances productivity and control, two requirements that are difficult to satisfy in contexts that aim to use AI directly in the data dimension.

Bibliography

- [1] IBM. *What is metadata management?* 2026. URL: <https://www.ibm.com/think/topics/metadata-management> (cit. on pp. 1, 2, 74).
- [2] IBM. *What is data lineage?* 2026. URL: <https://www.ibm.com/think/topics/data-lineage> (cit. on pp. 1, 3, 74, 79).
- [3] Snowflake. *Snowflake key concepts and architecture*. 2026. URL: <https://docs.snowflake.com/en/user-guide/intro-key-concepts> (cit. on pp. 1, 3, 27, 77, 86).
- [4] dbt Labs. *What is dbt?* 2026. URL: <https://docs.getdbt.com/docs/introduction> (cit. on pp. 1, 3, 77, 86).
- [5] DataHub Project. *Introduction to Metadata Ingestion*. 2026. URL: <https://docs.datahub.com/docs/metadata-ingestion> (cit. on pp. 1, 19, 46, 48, 77, 78, 86).
- [6] Microsoft. *Data lineage in Power BI*. 2025. URL: <https://learn.microsoft.com/en-us/power-bi/collaborate-share/service-data-lineage> (cit. on pp. 1, 3, 46, 77).
- [7] Model Context Protocol. *What is the Model Context Protocol (MCP)?* 2026. URL: <https://modelcontextprotocol.io/introduction> (cit. on pp. 1, 4, 57, 61).
- [8] Target Reply. *Target Reply - Chi siamo*. 2026. URL: <https://www.reply.com/target-reply/it/> (cit. on p. 2).
- [9] DataHub Project. *Introduction - The #1 Open Source AI Data Catalog*. 2026. URL: <https://docs.datahub.com/docs/introduction> (cit. on pp. 3, 10, 48, 50, 79).
- [10] DataHub Project. *DataHub Architecture Overview*. 2026. URL: <https://docs.datahub.com/docs/architecture/architecture> (cit. on pp. 4, 8, 11).
- [11] Acryl Data. *Context Management Is the Missing Piece in the Agentic AI Puzzle*. 2026. URL: <https://datahub.com/blog/context-management-is-the-missing-piece-in-the-agentic-ai-puzzle/> (cit. on pp. 4, 57, 82).

- [12] DataHub Project. *DataHub Components Overview*. 2026. URL: <https://docs.datahub.com/docs/components> (cit. on pp. 8, 11, 19).
- [13] DataHub Project. *DataHub APIs and SDKs Overview*. 2026. URL: <https://docs.datahub.com/docs/api/datahub-apis> (cit. on pp. 10, 22).
- [14] DataHub Project. *Metadata Model*. 2026. URL: <https://docs.datahub.com/docs/metadata-modeling/metadata-model> (cit. on pp. 14–17, 50, 51).
- [15] DataHub Project. *Metadata Events (MCP, MCL, Platform Events)*. 2026. URL: <https://docs.datahub.com/docs/what/mxe> (cit. on pp. 18, 19).
- [16] DataHub Project. *Metadata Ingestion Architecture*. 2026. URL: <https://docs.datahub.com/docs/architecture/metadata-ingestion> (cit. on pp. 19–21, 85).
- [17] DataHub Project. *DataHub GraphQL API Overview*. 2026. URL: <https://docs.datahub.com/docs/api/graphql/overview> (cit. on pp. 22, 50, 56).
- [18] DataHub Project. *DataHub MCP Server*. 2026. URL: <https://docs.datahub.com/docs/features/feature-guides/mcp> (cit. on pp. 46, 61, 62, 82).
- [19] Acryl Data. *DataHub MCP Server (GitHub repository)*. 2026. URL: <https://github.com/acryldata/mcp-server-datahub> (cit. on pp. 46, 61, 62).
- [20] DataHub Project. *Databricks Source (Unity Catalog) - DataHub Ingestion*. 2026. URL: <https://docs.datahub.com/docs/generated/ingestion/sources/databricks> (cit. on p. 48).
- [21] Vijay Khatrı and Carol V. Brown. «Designing Data Governance». In: *Communications of the ACM* 53.1 (2010), pp. 148–152. DOI: 10.1145/1629175.1629210 (cit. on pp. 50, 70, 85, 87).
- [22] IBM. *What is data governance?* 2026. URL: <https://www.ibm.com/think/topics/data-governance> (cit. on pp. 50, 70, 82, 85).
- [23] DataHub Project. *Assertion (DataHub Metamodel Entity)*. 2026. URL: <https://docs.datahub.com/docs/generated/metamodel/entities/assertion> (cit. on pp. 56, 81).
- [24] DataHub Project. *Incident (DataHub Metamodel Entity)*. 2026. URL: <https://docs.datahub.com/docs/generated/metamodel/entities/incident> (cit. on pp. 56, 81).
- [25] Model Context Protocol. *Model Context Protocol Specification (latest)*. 2026. URL: <https://modelcontextprotocol.io/specification/latest> (cit. on pp. 57–60, 85, 87).
- [26] Anthropic. *Introducing the Model Context Protocol*. 2024. URL: <https://www.anthropic.com/news/model-context-protocol> (cit. on pp. 57, 61).

- [27] Model Context Protocol. *Architecture overview*. 2026. URL: <https://modelcontextprotocol.io/docs/learn/architecture> (cit. on pp. 58–60).
- [28] Model Context Protocol. *Example Clients*. 2026. URL: <https://modelcontextprotocol.io/clients> (cit. on p. 58).
- [29] Anthropic. *Building effective agents*. 2024. URL: <https://www.anthropic.com/research/building-effective-agents> (cit. on pp. 71, 81, 85, 87).